

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет «Школа информатики, физики и технологий»

[Фамилия Имя Отчество автора]

**Адаптивные топологии мульти-агентных
систем больших языковых моделей с включением
человека в контур управления**

Выпускная квалификационная работа

бакалаврская работа

по направлению подготовки 01.03.02

«Прикладная математика и информатика»

образовательная программа

«Прикладной анализ данных и искусственный интеллект»

Рецензент

Руководитель

[уч. степень, звание]

[уч. степень, звание]

[И.О. Фамилия]

[И.О. Фамилия]

Санкт-Петербург, 2026

Содержание

Аннотация	4
Abstract	4
Ключевые слова	5
Введение	6
1 Аналитический обзор литературы	10
2 Постановка задачи и методология исследования	16
3 Архитектура программной системы	27
4 Экспериментальное исследование	34
5 Анализ результатов и обсуждение	51
Авторский вклад	55
Заключение	58
Библиографический список	60
А Функциональная схема системы	63
Б Глоссарий	66
В Состав программного репозитория	68
Г Открытый исходный код	69

Аннотация

В работе проведена количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации и позиции человека в контуре управления на эффективность решения задач разных классов. Разработан фреймворк под лицензией MIT, объединяющий пять статических топологий, адаптивную мета-топологию, пять ролей человека и два режима маршрутизации; воронка из четырех экспериментов с двумя кросс-семейными подтверждающими запусками охватывает 4230 завершенных запусков на четырех корпусах задач. RQ1 и RQ3 подтверждены. RQ2 раскладывается на task-difficulty-conditional Парето-преимущество адаптивной мета-топологии на сложных корпусах и routing strategy как per-family deployment decision — Парето-победитель меняется при смене семейства весов работника ($\Delta q^{\text{rule-llm}} = +0,304$ на Qwen-3-235B, бутстрап-CI $[+0,185, +0,426]$ исключает нуль). RQ4 опровергнут как universal negative finding — адаптивный маршрутизатор роли не дает положительного лифта ни на одном из двух семейств работника. Центральный structural finding работы — зависимость выбора routing-стратегии от семейства весов рабочего агента, требующая обязательной кросс-семейной валидации.

Abstract

The thesis provides a quantitative assessment of how the choice of communication topology, its runtime adaptation, and the position of the human in the control loop affect the effectiveness of multi-agent large-language-model systems on tasks of different classes. A framework was developed under the MIT license, integrating five static topologies, an adaptive meta-topology, five human roles, and two topology routing modes; a funnel of four experiments with two cross-family confirmation runs covers 4230 completed runs across four task corpora. RQ1 and RQ3 are confirmed. RQ2 decomposes into a task-difficulty-conditional Pareto ad-

vantage of the adaptive meta-topology on hard corpora and routing strategy as a per-family deployment decision — the Pareto winner flips when the worker family changes ($\Delta q^{\text{rule-llm}} = +0.304$ on Qwen-3-235B, bootstrap CI $[+0.185, +0.426]$ robustly excludes zero). RQ4 is rejected as a universal negative finding — the adaptive role router yields no positive lift on either worker family. The central structural finding is the dependence of routing strategy selection on the weight family of the worker agent, which requires mandatory cross-family validation.

Ключевые слова

мульти-агентные системы, большие языковые модели, коммуникационная топология, адаптивная маршрутизация, человек в контуре управления, per-task best-of референс, кросс-семейная валидация, Парето-оптимизация, бутстрап-оценивание, оркестрация LLM-агентов.

Keywords: multi-agent systems, large language models, communication topology, adaptive routing, human-in-the-loop, per-task best-of baseline, cross-family validation, Pareto optimization, bootstrap inference, LLM agent orchestration.

Введение

Актуальность работы. В 2023–2026 годах применение больших языковых моделей (от англ. Large Language Models, LLM) перешло от одиночных диалоговых сценариев к кооперативной работе нескольких автономных агентов, образующих мульти-агентную систему (от англ. Multi-Agent System, MAS). Открытые фреймворки (AutoGen, ChatDev, MetaGPT, Magentic-One) показывают, что разделение труда между специализированными агентами повышает качество решения сложных задач, но каждый из них жестко фиксирует структуру коммуникации на этапе проектирования; систематических данных об оптимальной топологии для каждого класса задач в открытой литературе нет. Параллельный интерес к интеграции человека в контур управления ограничивается единичными паттернами без сравнения эффективности. Сочетание этих обстоятельств определяет научную и прикладную актуальность работы.

Научная новизна. В настоящей работе впервые проведено прямое сравнение пяти базовых коммуникационных топологий (звезда, цепочка, полносвязная, дебатная, иерархическая) на едином программном стенде, на одной языковой модели и на одной выборке задач из четырех различных классов. Предложена и реализована мета-топология, переключающая активную базовую конфигурацию в зависимости от текущей фазы решения и наблюдаемых сигналов агентов. Введена таксономия из пяти специфичных для адаптивных мульти-агентных систем метрик — N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle} , — позволяющая количественно характеризовать поведение мета-топологии. Впервые сформулирована и экспериментально проверена матрица совместимости пяти ролей человека в контуре управления с пятью базовыми топологиями.

Предмет исследования. Коммуникационная топология мульти-агентной системы больших языковых моделей, механизм ее динамической адаптации в процессе решения задачи, а также позиция человека в графе обмена сообщениями между агентами.

Объект исследования. Мульти-агентные системы на основе больших

языковых моделей, решающие сложные задачи различных классов в кооперации со специализированными агентами и участником-человеком.

Цель работы. Количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации и позиции человека в контуре управления на эффективность решения задач разных классов с одновременным учетом качества решения, его стоимости и времени выполнения.

Аналитический обзор предлагаемых решений. В 2022–2026 годах выделяются три направления развития MAS LLM. Архитектурные фреймворки (AutoGen, ChatDev, MetaGPT) фиксируют топологию и не сравнивают альтернативы. Автоматическое проектирование графов (G-Designer, GPTSwarm) подбирает структуру однократно и не пересматривает ее. Динамическая адаптация (DyLAN, AgentVerse) меняет состав команды, но не структуру связей. Вопрос роли человека освещен фрагментарно, без количественной оценки ролей. Подробный анализ — в главе 1, сравнительная таблица 1.1 восемнадцати методов по шести критериям.

Практическая значимость. Работа дает разработчикам MAS количественные данные о том, какая топология эффективнее для каждого из четырех рассмотренных классов задач и когда имеет смысл адаптивное переключение. Программная инфраструктура реализована как открытый фреймворк адаптивных топологий (от англ. Adaptive Topologies for Multi-agent systems, ATM), исходный код — в репозитории GitHub (приложение Г), допускает повторное использование и расширение. Сформулированные правила выбора топологии и роли человека под класс задачи применимы при проектировании интеллектуальных ассистентов в программировании, аналитике и принятии решений.

Теоретическая значимость. Введенная таксономия адаптивно-специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle}) позволяет количественно характеризовать поведение динамически перестраиваемых MAS по единой шкале. Формализация MAS как кортежа из шести компонентов с фазово-зависимой коммуникационной структурой расширяет теоретическую базу; эмпирическая

проверка гипотезы о фазовой специфике топологий дает обоснование для дальнейших теоретических работ.

Методы исследования. Комплекс взаимодополняющих методов: аналитический обзор литературы по ключевым словам в открытых базах; формализация MAS средствами теории графов и конечных автоматов; программная реализация на Python с LangGraph; контролируемый эксперимент с предварительной регистрацией гипотез и порогов улучшения; статистическая обработка (дисперсионный анализ от англ. Analysis of Variance, ANOVA; парные сравнения с поправкой Бенджамини–Хохберга на уровне ложных открытий от англ. False Discovery Rate, FDR; коэффициент Коэна; непараметрический бутстрап с доверительными интервалами от англ. Confidence Interval, CI); оракульный референс per-task best-of для верхней границы качества адаптации; имитационное моделирование участника-человека языковой моделью с подсказкой по роли.

Задачи работы.

1. Провести аналитический обзор работ 2022–2026 годов по мульти-агентным системам больших языковых моделей и интеграции человека в контур управления, выделить исследовательскую лакуну.
2. Сформулировать формальную модель MAS как кортежа из множеств агентов, ребер коммуникации, ролей агентов и человека, задачи и фаз решения.
3. Разработать программный фреймворк, объединяющий пять базовых топологий и адаптивную мета-топологию на базе LangGraph.
4. Спроектировать систему оценочных метрик — базовые показатели качества, стоимости и времени и специфичные для адаптивной мета-топологии характеристики (переключения, защитные блокировки, стоимостной вклад маршрутизатора, доля регрессий, разрыв до верхней границы).
5. Провести воронку из четырех экспериментов и кросс-семейных подтверждающих запусков на четырех классах задач — программирование, математические рассуждения, творческая генерация, анализ дан-

ных — с предзафиксированными гипотезами и порогами.

6. Выполнить статистический анализ результатов и сформулировать ответы на четыре исследовательских вопроса (от англ. Research Question, RQ), описанных в разделе 2.7.
7. Выработать практические рекомендации по выбору топологии и роли человека в зависимости от класса задачи.

Структура работы. Работа состоит из введения, пяти глав основного содержания, раздела авторского вклада, заключения, библиографического списка и четырех приложений. Глава 1 — аналитический обзор литературы и исследовательская лакуна; Глава 2 — формальная модель, шесть топологий, пять ролей человека, метрики, корпус, дизайн воронки; Глава 3 — архитектура программной системы; Глава 4 — результаты четырех экспериментов и двух кросс-семейных подтверждающих запусков; Глава 5 — анализ, ответы на четыре RQ, угрозы валидности. Приложения — функциональная схема, глоссарий, состав репозитория, ссылка на исходный код. В работе 21 таблица, 7 рисунков и 5 формул; общий объем с приложениями — 69 страниц; библиографический список — 44 источника.

1 Аналитический обзор литературы

Глава охватывает пять направлений — архитектуру мульти-агентных систем больших языковых моделей, коммуникационные топологии, механизмы их адаптации, безопасность и интеграцию человека в контур управления — и завершается сравнительной таблицей методов и определением исследовательской лакуны.

В выборку включены работы 2022–2026 годов, отобранные по ключевым словам *multi-agent LLM*, *agent topology*, *LLM debate*, *human-in-the-loop multi-agent* в базах arXiv, ACL Anthology, ICLR, NeurIPS, ICML. Из более чем шестидесяти работ оставлены те, которые либо предлагают новую архитектуру взаимодействия агентов, либо систематически анализируют свойства существующих.

1.1 Мульти-агентные системы больших языковых моделей

С появлением больших языковых моделей (от англ. Large Language Models, LLM) концепция мульти-агентных систем (от англ. Multi-Agent System, MAS) переживает второе рождение — роль агента теперь играет языковая модель с системной подсказкой и набором инструментов. Обзор [1] систематизирует быстрорастущую совокупность исследований и выделяет несколько устойчивых архитектурных образцов.

Фреймворк AutoGen [4] реализует многораундовый диалог агентов с вызовом инструментов и подключением человека, но структура взаимодействия определяется императивным кодом сценария и не сравнивается между конфигурациями. ChatDev [8] моделирует разработку ПО как последовательность ролевых агентов (директор, программист, тестировщик) по водопадной модели; MetaGPT [31] развивает идею через стандартные операционные процедуры. В обеих системах топология жестко зашита в архитектуру. Одно-агентные методы Self-Refine [41] и Reflexion [38] реализуют цикл «генерация — критика — переработка» с вербальным подкреплением, но не используют разнообразия точек зрения мульти-агентных конфигураций.

1.2 Коммуникационные топологии агентов

Под коммуникационной топологией понимается граф, описывающий, какие агенты могут обмениваться сообщениями и в каком порядке. Таксономия настоящей работы включает пять базовых типов — звезда, цепочка, полносвязная, дебатная, иерархическая — изображенных на рисунке 1.1.

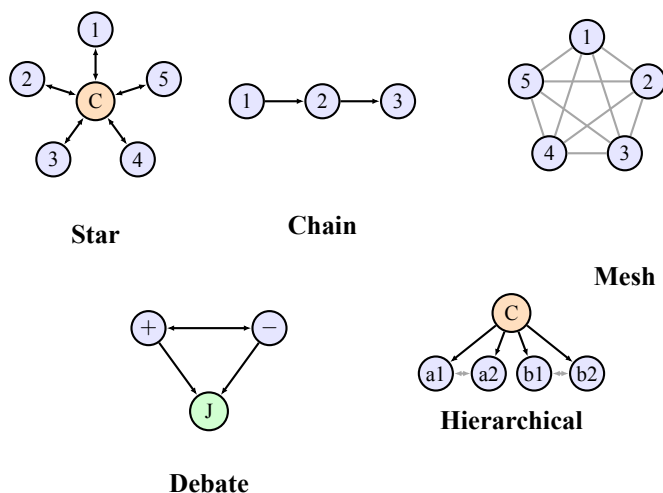


Рисунок 1.1. Схематическое изображение пяти базовых коммуникационных топологий, рассматриваемых в работе. Узлы — агенты; C — координатор, J — арбитр, + и — — оппоненты в дебатах, a1, a2, b1, b2 — участники подкоманд. Сплошные направленные ребра обозначают передачу управления, серые двунаправленные — обмен сообщениями в общей шине.

GPTSwarm [17] рассматривает мульти-агентную систему как обучаемый граф вычислений и оптимизирует его параметры на корпусе задач, но пространство поиска ограничено статичными структурами. G-Designer [16] применяет графовые нейронные сети (от англ. Graph Neural Network, GNN) для подбора топологии под задачу, но выбор производится однократно. В [40] показано, что топологии типа малого мира (small-world) повышают робастность, но сравнение ведется только между статичными конфигурациями. DyLAN [28] варьирует состав команды, оставляя структуру связей фиксированной; Mixture-of-Agents [32] реализует многослойную ансамблевую конфигурацию без изменения во время решения.

Tree-of-Thoughts [44] разворачивает дерево вариантов рассуждения, CAMEL [6] моделирует ролевой диалог двух агентов; дебаты языковых моделей [21] по-

вышают качество вывода через обмен аргументами. Во всех этих методах структура взаимодействия фиксирована на протяжении задачи. Возможность переключения топологии в процессе решения в этих работах не рассматривается.

1.3 Механизмы адаптации топологий

Адаптация мульти-агентных систем во время выполнения исследуется реже. Преобладают два направления, ни одно из которых не покрывает динамическое переключение топологии между фазами.

Первое направление — адаптация состава команды. DyLAN [28] выбирает на каждом шаге подмножество агентов; AgentVerse [3] использует механизм найма из заранее заданного пула. Изменяется множество участников, а не схема обмена. Второе направление — однократная офлайн-оптимизация структуры графа перед запуском задачи (GPTSwarm [17], G-Designer [16]).

Сложные задачи неоднородны по структуре — фаза планирования требует генерации альтернатив, исполнения — иерархического разделения труда, проверки — независимой рецензии. Отсюда гипотеза: статическая топология, оптимальная для усредненной картины, может быть субоптимальной для каждой отдельной фазы. Эта гипотеза мотивирует разработку механизма переключения топологии в процессе выполнения, что и составляет один из центральных предметов исследования.

1.4 Безопасность и устойчивость топологий

NetSafe [35] показывает, что плотные топологии (полносвязная, звезда) более подвержены каскадному распространению ошибок при компрометации агента, чем разреженные (цепочка). ТОМА [42] развивает анализ на многошаговые атаки с учетом связности. Эти результаты мотивируют включение в систему защитных правил переключения топологий, предотвращающих патологические режимы (в частности, осцилляционное переключение без прогресса); их реализация описана в главе 3.

1.5 Человек в контуре управления

Подключение человека (от англ. Human-in-the-Loop, HITL) систематизировано в [34] в части разметки, исправления и активного отбора; классическая работа [20] ввела принципы смешанной инициативы. Методы выравнивания моделей с обратной связью от человека ([10], InstructGPT [36]) ставят человека в роль поставщика сигнала вознаграждения — в MAS вопрос принципиально иной — роль человека в каждом вызове требует формализации в графе сообщений.

Обзор [1] перечисляет паттерны (координатор, рецензент, судья), но без сравнения эффективности. AutoGen [4] допускает человека как агента, но роль задается архитектурно. Magentic-One [29] разделяет координатора и работников с опциональным человеком-постановщиком задачи, также без сравнения ролей. Шкала NASA-TLX [18] применяется в оценке когнитивной нагрузки, но не пересекается с проектированием графа агентов.

В выборке 2022–2026 годов систематического сравнения ролей человека и фаз их активации не обнаружено — эта лакуна составляет второй центральный предмет исследования.

1.6 Сравнительная аналитическая таблица методов

Сопоставление методов по ключевым критериям приведено в таблице 1.1. Знаки «+» / «−» / «±» означают наличие, отсутствие и частичную реализацию свойства.

1.7 Что отсутствует в существующих работах

Три столбца таблицы 1.1 — систематическое сравнение топологий, адаптация во время решения и формализованная роль человека — в выборке не встречаются одновременно. G-Designer [16] сравнивает топологии, но не адаптирует их в процессе; AutoGen [4] подключает человека без формализации и сравнения ролей; DyLAN [28] адаптирует состав, а не структуру связей.

Заполнение этой лакуны составляет цель работы. Предлагаются — во-

Таблица 1.1. Сравнение существующих методов по ключевым критериям

Метод	Год	MAS	Сравн. топ.	Адапт. состава	Адапт. топ. runtime	HITL	Безопасн.
ReAct [37]	2022	—	—	—	—	—	—
AgentVerse [3]	2023	+	—	+	—	—	—
LLM debate [21]	2023	+	—	—	—	—	—
CAMEL [6]	2023	+	—	—	—	—	—
DyLAN [28]	2023	+	—	+	—	—	—
Reflexion [38]	2023	—	—	—	—	—	—
AutoGen [4]	2023	+	—	—	—	+	—
Tree-of-Thoughts [44]	2023	—	—	—	—	—	—
Magentic-One [29]	2024	+	—	—	—	±	—
MetaGPT [31]	2024	+	—	—	—	—	—
TOMA [42]	2024	+	±	—	—	—	+
Self-Refine [41]	2024	—	—	—	—	—	—
ChatDev [8]	2024	+	—	—	—	—	—
Scaling MAS [40]	2024	+	±	—	—	—	±
MoA [32]	2024	+	—	—	—	—	—
NetSafe [35]	2024	+	+	—	—	—	+
G-Designer [16]	2024	+	+	—	—	—	—
GPTSwarm [17]	2024	+	±	—	—	—	—
Настоящая работа	2026	+	+	—	+	+	±

Обозначения колонок — **MAS** мульти-агентная архитектура, **Сравн. топ.** систематическое сравнение нескольких топологий на едином стенде, **Адапт. состава** динамическое изменение участников, **Адапт. топ. runtime** переключение структуры графа в процессе решения задачи, **HITL** интеграция человека в контур управления как полноценного участника графа, **Безопасн.** учет устойчивости к скомпрометированному агенту и каскадным сбоям. Знак «+» означает наличие свойства, «—» — его отсутствие, «±» — частичную реализацию.

первых, единый стенд для сравнения пяти топологий на четырех классах задач; во-вторых, мета-топология с переключением активной базовой по фазе решения; в-третьих, формализация пяти ролей человека с экспериментальной оценкой по факторному плану.

1.8 Выводы и результаты по главе

Коммуникационная топология в существующих MAS фиксируется на этапе проектирования; механизмы адаптации ограничены составом команды или однократным офлайн-выбором структуры. Безопасность существенно зависит от топологии, что мотивирует защитные правила при ее динамическом изменении. Систематического исследования роли человека в графе агентов не обнаружено; смежные работы по бенчмаркингу (AgentBench [2]) и маршрутизации между моделями (RouteLLM [39], FrugalGPT [9]) формализованной роли человека также не вводят. На пересечении трех направлений

— сравнения, адаптации, роли человека — лежит лакуна, формализуемая четырьмя исследовательскими вопросами в разделе 2.

2 Постановка задачи и методология исследования

Глава формализует объект исследования, исследовательские вопросы и методологию их эмпирической проверки — математическая модель, шесть топологий, пять ролей человека, система метрик, обоснование корпуса и моделей, воронка из четырех экспериментов.

2.1 Формальная модель мульти-агентной системы

Мульти-агентная система рассматривается как кортеж

$$\mathcal{M} = \langle V, E_t, R, \mathcal{T}, F, h \rangle, \quad (2.1)$$

где $V = \{v_1, \dots, v_n\}$ — конечное множество агентов, каждый из которых реализован как языковая модель с фиксированной системной подсказкой и набором инструментов; $E_t \in 2^{V \times V}$ — множество допустимых направленных ребер коммуникации на итерации t , образующее коммуникационный граф; $R: V \rightarrow \mathcal{R}_A$ — отображение агентов в множество ролей агентов $\mathcal{R}_A$ из шести значений (планировщик, исследователь, исполнитель, критик, дебатер, координатор); $\mathcal{T}$ — текущая задача с целевой функцией оценки качества решения; F — конечное множество из четырех фаз решения задачи (планирование, исполнение, проверка, завершение); h — (опционально) участник-человек с ролью из отдельного множества $\mathcal{R}_H$ (см. 2.3).

В классических работах граф E_t определяется архитектурно и не меняется в ходе выполнения задачи. В настоящем исследовании вводится расширение модели — структура E_t становится результатом оператора обновления, зависящего от текущей фазы $f_t \in F$ и наблюдаемых сигналов $\sigma_t \in \{0, 1\}^m$ агентов:

$$E_t = \Phi(f_t, \sigma_t, t), \quad (2.2)$$

где Φ — оператор обновления структуры коммуникационного графа, m —

фиксированная размерность вектора сигналов (флаги завершения этапа, признаки тупикового состояния, счетчики отказов критика и другие наблюдаемые индикаторы прогресса). Возможность такого переключения и составляет суть исследуемой адаптивной мета-топологии.

Множество ролей агентов $\mathcal{R}_A$ выше уже зафиксировано шестью значениями. Множество ролей человека $\mathcal{R}_H$ фиксировано и содержит пять значений, формализованных в разделе 2.3. Фазы F упорядочены отношением $planning \prec execution \prec verification \prec done$ с монотонной семантикой переходов.

Задача оптимизации мульти-агентной системы формулируется как многокритериальная по двум первичным осям — качеству решения $q(\mathcal{M})$ и суммарной стоимости вызовов $c(\mathcal{M})$ — при ограничении на время выполнения $\tau(\mathcal{M}) \leq \tau_{\max}$, где $\tau_{\max}$ задается конфигурацией эксперимента. Поскольку чистый Парето-оптимум обычно не единственен, в работе используется скаляризация с фиксированными порогами улучшения по двум первичным осям. Конфигурация $\mathcal{M}_1$ считается выигрышной относительно $\mathcal{M}_0$, если соблюдено ограничение $\tau(\mathcal{M}_1) \leq \tau_{\max}$ и выполняется одно из условий

$$\frac{q(\mathcal{M}_1) - q(\mathcal{M}_0)}{q(\mathcal{M}_0)} \geq 0,05 \quad \text{или} \quad \frac{c(\mathcal{M}_0) - c(\mathcal{M}_1)}{c(\mathcal{M}_0)} \geq 0,15 \quad \text{при} \quad q(\mathcal{M}_1) \geq q(\mathcal{M}_0). \quad (2.3)$$

Пороги 5% для качества и 15% для стоимости зафиксированы до проведения основных запусков и обоснованы в разделе 2.4.

2.2 Шесть рассматриваемых топологий

Пространство сравнения составляют шесть топологий. Пять из них являются статичными и реализуют пять базовых классов коммуникационных структур, перечисленных в обзоре литературы. Шестая является мета-топологией, переключающей активную статическую топологию в зависимости от фазы.

Star — центральный координатор v_c последовательно обращается к специализированным работникам $v_1, \dots, v_k$ и агрегирует их ответы. Граф ребер $E_{\text{star}} = \{(v_c, v_i), (v_i, v_c) \mid i = 1, \dots, k\}$.

Chain — линейный конвейер из трех агентов $v_p \rightarrow v_e \rightarrow v_c$ (планировщик, исполнитель, критик) с условным циклом доработки. Граф направленный ациклический с обратной связью от критика к исполнителю.

Mesh — полносвязная топология с общей шиной сообщений. Граф $E_{\text{mesh}} = (V \times V) \setminus \{(v, v)\}$, обмен организован по принципу циклической очереди, решение принимается голосованием.

Debate — пара оппонентов v_+ и v_- генерирует параллельные альтернативные решения, после чего судья v_j выбирает победителя или синтезирует ответ. Граф двудольный с агрегатором.

Hierarchical — двухуровневая иерархия из координатора верхнего уровня v_c^{top} и двух подкоманд $T_a, T_b \subset V$, каждая из которых сама по себе является графом. Подкоманды работают параллельно, координатор синтезирует итог.

Adaptive — мета-топология, активирующая на каждой итерации одну из пяти базовых конфигураций. Решение о выборе принимается маршрутизатором согласно формуле (2.2). Псевдокод процедуры приведен в таблице 2.1, общая концептуальная схема взаимодействия блоков системы изображена на рисунке 3.1 главы 3, архитектура реализации описана в разделе 3.3.

Таблица 2.1. Процедура работы адаптивной мета-топологии

Шаг	Действие
1	Инициализировать фазу $f \leftarrow \text{planning}$, начальную топологию $K \leftarrow K_0$, состояние $s \leftarrow s_0$ и журнал $\mathcal{J} \leftarrow \emptyset$.
2	Пока $f \neq \text{done}$ и $c(s) < B$ выполнять шаги 3–10.
3	Применить маршрутизатор фаз $f' \leftarrow \text{PhaseRouter}(s, f)$.
4	Применить маршрутизатор топологий $K' \leftarrow \text{TopologyRouter}(s, K, f')$.
5	Если $\text{Guard}(s, K, K') = \text{block}$, то $K' \leftarrow K$ и записать guard_override в $\mathcal{J}$.
6	Если $K' \neq K$, применить ворота перехода состояния $s \leftarrow \text{TransitionGate}(s, K, K')$ и записать переход в $\mathcal{J}$.
7	Исполнить подграф выбранной топологии $s \leftarrow \text{Subgraph}_{K'}(s)$.
8	Обновить $K \leftarrow K'$ и $f \leftarrow f'$.
9	Записать журнал итерации в $\mathcal{J}$.
10	Перейти к шагу 2.
11	Вернуть результат $y(s)$ и журнал $\mathcal{J}$.

2.3 Пять ролей человека в контуре управления

Пять ролей человека формализованы по аналогии с типовыми позициями в человеческих организациях. **Координатор** (ρ_C) — управляет планом и делегированием, активен в иерархических и полносвязных топологиях. **Рецензент** (ρ_R) — проверяет промежуточные результаты и инициирует доработку, естественно вписывается в Star и Chain. **Судья** (ρ_J) — выносит финальный вердикт при наличии альтернатив, релевантен для Debate. **Равноправный участник** (ρ_P) — вносит альтернативные мнения наравне с агентами, вписывается в Mesh. **Наблюдатель** (ρ_M) — следит за процессом и вмешивается только в критических случаях (превышение бюджета, зависание, циклы), применим в любой топологии.

Не все пары (топология, роль) являются осмысленными. Рассматриваются пять статических топологий и пять ролей человека — полное произведение содержит 25 пар. Из него исключаются три заведомо вырожденные комбинации. Пара (Debate, Coordinator) исключается, поскольку координатор в дебатной топологии эквивалентен судье. Пары (Chain, Peer) и (Hierarchical, Peer) исключаются, поскольку равноправный участник ломает линейную или иерархическую структуру. После исключений пространство содержит $5 \times 5 - 3 = 22$ осмысленные комбинации топология $\times$ роль человека, где используются пять статических топологий и пять ролей человека из $\mathcal{R}_H$.

2.4 Система оценочных метрик

Метрики разделены на три группы. Первая группа — базовые метрики качества и эффективности — применима ко всем экспериментам и ко всем топологиям.

- $q \in [0, 1]$ — агрегированный показатель качества решения. Конкретная формула зависит от класса задач — для задач программирования это метрика `pass@1` (доля задач, для которых сгенерированный код проходит все предзаданные модульные тесты с первой попытки); для математических задач — `exact match` (точное совпадение числового

ответа с эталоном); для творческой генерации — полусумма метрики ROUGE-L [26] (от англ. Recall-Oriented Understudy for Gisting Evaluation, Longest common subsequence variant) и доли покрытых концептов; для анализа данных — exact match числового ответа.

- c — суммарная стоимость вызовов языковых моделей в долларах США за один запуск.
- τ — время выполнения запуска по настенным часам (от англ. wall-clock) в секундах.
- r_{fail} — доля запусков, завершившихся со статусом, отличным от успешно выполненного.
- N_{iter} — число итераций основного цикла.

Вторая группа — метрики, специфичные для адаптивной мета-топологии.

- N_{switch} — число фактических переключений активной топологии за один запуск.
- r_{guard} — доля решений маршрутизатора, заблокированных защитными правилами.
- γ — доля бюджета, потраченная маршрутизатором на собственные вызовы языковой модели; критичная метрика, отвечающая на вопрос, не превышает ли стоимость адаптации получаемый от нее выигрыш.
- r_{hurt} — доля задач, на которых адаптивная конфигурация дает качество хуже лучшей статической конфигурации. Является ключевым индикатором безопасности механизма адаптации.
- Δ_{oracle} — разрыв качества между теоретическим per-task best-of референсом и фактическим маршрутизатором, формально определяемый по формуле (2.4). Per-task best-of референс вычисляется по результатам базового эксперимента E1 — для каждого корпуса задач выбирается статическая топология с максимальным средним качеством на этом корпусе. Референс служит верхней границей достижимого качества для класса задач при заданном пространстве статических топологий и требует апостериорного знания пары (корпус, лучшая топология); в открытом deployment’е без классификатора задач недостижим.

$$\Delta_{\text{oracle}} = q_{\text{best-of}} - q_{\text{router}}, \quad \text{где} \quad q_{\text{best-of}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} q(K_c^*, c), \quad (2.4)$$

где $\mathcal{C}$ — множество корпусов задач, $K_c^* = \arg \max_K q(K, c)$ — оптимальная статическая топология для корпуса c , $q(K, c)$ — среднее качество топологии K на корпусе c по базовому эксперименту E1.

Третья группа — метрики, относящиеся к взаимодействию с человеком.

- N_{int} — число обращений системы к человеку за один запуск.
- L_{cog} — прокси-метрика когнитивной нагрузки для запусков с симулятором человека, формально определяемая по формуле (2.5). В экспериментах с реальными участниками заменяется на стандартную шкалу NASA-TLX.

$$L_{\text{cog}} = w_n \cdot \tilde{N}_{\text{int}} + w_l \cdot \tilde{L}_{\text{ctx}} + w_\tau \cdot \tilde{\tau}_{\text{resp}}, \quad w_n = w_l = w_\tau = \frac{1}{3}, \quad (2.5)$$

где $\tilde{N}_{\text{int}}$, $\tilde{L}_{\text{ctx}}$ и $\tilde{\tau}_{\text{resp}}$ — нормированные на отрезок $[0, 1]$ значения числа обращений к участнику, средней длины предоставляемого ему контекста и средней задержки ответа симулятора соответственно. Нормировка выполняется по квантилям распределения наблюдаемых значений за все запуски эксперимента. Равные веса выбраны априорно до начала запусков как нейтральная свертка трех независимых компонентов нагрузки.

Адаптивная конфигурация считается выигрышной относительно наилучшей статической, если она дает прирост качества не менее 5% либо снижение стоимости не менее 15% при сохранении качества. Пороги фиксируются предварительно, до проведения основных запусков, что соответствует практике предварительной регистрации гипотез в эмпирических исследованиях.

2.5 Корпус задач и обоснование его выбора

Эксперименты проводятся на стратифицированной выборке из четырех открытых корпусов, покрывающих четыре качественно различных класса за-

дач — программирование, математические рассуждения, творческая генерация и анализ данных. Сравнение корпуса с альтернативами приведено в таблице 2.2.

Таблица 2.2. Сравнение рассмотренных корпусов задач

Корпус	Класс	Размер	Метрика	Решение по включению
HumanEval [15]	Программирование	164	pass@1	Включен; стандарт сравнения.
HumanEval+ [23]	Программирование	164	pass@1+	Не включен; превышает покрытие основной выборки.
LiveCodeBench [27]	Программирование	500+	pass@1	Резерв для проверки на загрязнение.
GSM8K [43]	Математ. рассуждения	8500	exact match	Включен; стандарт оценки.
MATH [30]	Математ. рассуждения	12500	exact match	Не включен; сложность выше возможностей рабочих моделей.
MMLU-Pro [33]	Множеств. рассужд.	12000	accuracy	Не включен; формат ответа не подходит для дебатовой топологии.
CommonGen [11]	Творческая генерация	35000	ROUGE-L + cov	Включен; покрывает open-ended генерацию с ограничениями.
InfiAgent-DABench [22]	Анализ данных	257	numeric exact	Включен; единственный корпус с замкнутым ответом по аналитике данных.
HellaSwag [19]	Здравый смысл	10000	accuracy	Качество на корпусе model-sensitive (см. 4.6). Не включен; качество современных моделей близко к потолку.

Выбор обусловлен совокупностью требований. Каждый класс задач представлен ровно одним корпусом, чтобы избежать смещения эффектов корпуса и класса. Каждый корпус допускает автоматическую объективную оценку, исключая субъективность оценщика (юнит-тесты, точное совпадение числа, метрики покрытия концептов). Размер задач в каждом корпусе позволяет уложиться в один запуск рабочих моделей. От рассматриваемых для каждого класса альтернатив выбранные корпуса отличаются распространенностью в литературе и зрелостью методики оценки, что обеспечивает сопоставимость результатов с предшествующими работами. Из каждого корпуса производится стратифицированная выборка по 15 задач, итого 60 задач в одном базовом запуске. Каждая задача повторяется с тремя различными зерна-

ми генерации (для контроля стохастичности выборки токенов), что в комбинации с пятью статическими топологиями дает $60 \times 3 \times 5 = 900$ запусков в эксперименте E1.

2.6 Языковые модели и инфраструктура

Распределение моделей по ролям изолирует переменную «модель» от переменных «топология» и «роль человека». Все рабочие агенты (планировщик, исследователь, исполнитель, критик, дебатер), а также маршрутизаторы и симулятор человека используют единую модель Cerebras gpt-oss-120b (3000 токенов/с на Cerebras WSE). Иерархия «работник слабее критика» реализуется различиями в подсказках и параметре строгости рассуждения, а не разными моделями — фиксированная модельная мощность критична для чистоты сравнения топологий.

Судья оценки качества вынесен в отдельную линию — OpenAI gpt-4.1-mini с самосогласованностью из трех вызовов и нулевой температурой; модель из другого семейства весов исключает петлю положительной обратной связи. Подтверждающие запуски выполняются на Cerebras qwen-3-235b как представителе другого семейства весов.

Выбор моделей опирался на четыре критерия — отказоустойчивый программный интерфейс (от англ. Application Programming Interface, API) с вызовом инструментов и структурированным выводом, низкая удельная стоимость вызова, высокая пропускная способность и доступность из РФ.

Инфраструктура (глава 3) — LangGraph, PostgreSQL 16, Apache Parquet; параллелизм — 12 процессов с асинхронной обработкой внутри каждого.

2.7 Дизайн экспериментов — воронка

Прямой эксперимент по всему декартову произведению (6 топологий $\times$ 5 ролей $\times$ 3 режима $\times$ 4 класса задач) дал бы более 10 000 ячеек. Вместо этого исследование организовано как воронка из четырех экспериментов, каждый из которых сужает пространство конфигураций по результатам предыдущего (таблица 2.3).

Таблица 2.3. Структура экспериментальной воронки

Эксп.	Иссл. вопрос	Запуски
E1	RQ1	$5 \cdot 4 \cdot 15 \cdot 3 = 900$
E2	RQ3	≈ 2295 (после фильтра top-3)
E3	RQ2	≈ 640 (rule-full grid + четыре per-task llm-подсетки + три статические базовые)

Исследовательские вопросы формулируются следующим образом.

RQ1. Различаются ли пять статических топологий по количественным показателям качества, стоимости и времени для разных классов задач?

RQ2. Раскладывается на две взаимодополняющих части. (а) Дает ли адаптивная мета-топология значимый прирост качества или экономию стоимости по сравнению с per-task best-of референсом и с отдельными статическими топологиями? (б) Устойчив ли выбор оптимальной routing-стратегии (правиловой или языковой) к смене семейства весов рабочего агента, или он является per-family deployment decision?

RQ3. Какая из пяти ролей человека наиболее эффективна для каждой топологии и для каждого класса задач?

RQ4. Превосходит ли совместная адаптация топологии и роли человека по фазам решения фиксированную роль; сохраняется ли направление эффекта при смене семейства весов рабочего агента?

Статистическая обработка включает двухфакторный дисперсионный анализ (two-way ANOVA, тип III сумм квадратов для несбалансированного дизайна) с эффектом топологии и эффектом роли в качестве факторов и их взаимодействием на уровне значимости $\alpha = 0,05$. При нарушении допущений нормальности или равенства дисперсий применяется непараметрический аналог по критерию Краскела–Уоллиса. Парные сравнения средних выполняются с поправкой Бенджамини–Хохберга [5] на множественное сравнение при уровне ложных открытий $FDR = 0,05$. Оценка практической значимости различий проводится через коэффициент размера эффекта Коэна d [12]. Доверительные интервалы (от англ. Confidence Interval, CI) рассчитываются по методу непараметрического бутстрапа [14] (метод процентилей) с десятью тысячами ресэмплов на уровне 95 % при фиксированном начальном значении генератора псевдослучайных чисел.

2.8 Разграничение заимствованных и авторских элементов

Граница между заимствованными и авторскими элементами. К заимствованным относятся пять статических топологий, четыре открытых корпуса за-

дач, языковые модели и стандартный статистический инструментарий (ANOVA, бутстрап, коэффициент Коэна). К авторским — адаптивная мета-топология вместе с процедурой работы (2.2); формализация пяти ролей человека и матрица их совместимости с топологиями; набор адаптивно-специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle}); вороночная структура исследования с предварительной фиксацией порогов (2.3) и кросс-семейным подтверждением.

2.9 Выводы и результаты по главе

Формализована MAS как кортеж из шести компонентов с динамическим переключением графа по фазе; определены шесть топологий, пять ролей человека и три группы метрик (включая адаптивно-специфичные). Обоснован выбор корпуса из четырех бенчмарков и линии моделей с кросс-семейным судьей. Сформулированы четыре RQ и воронка из четырех экспериментов с подтверждающим запуском на дополнительном семействе моделей. Граница заимствованного и авторского зафиксирована в 2.8 и развернута в 5.5. План реализуется в инфраструктуре главы 3 и проверяется в 4.

3 Архитектура программной системы

Программная инфраструктура работы реализована как фреймворк адаптивных топологий для мульти-агентных систем (от англ. Adaptive Topologies for Multi-agent systems, ATM) на языке Python не ниже 3.11, распространяемый как пакет `atm`. Глава описывает архитектуру с акцентом на решения, относящиеся к научным гипотезам — адаптивному переключению топологий и интеграции человека. Исходный код — в приложении Г.

Функциональная схема — на рисунке 3.1. Вход — описание задачи и YAML-конфиг (от англ. YAML Ain't Markup Language); выход — журналы взаимодействия, метрики и записи о фазах, переключениях топологий и взаимодействиях с человеком. Между входом и выходом работают пять блоков — оркестратор, менеджер фаз и маршрутизаторы, активная топология с агентами, шлюз HITL и подсистема наблюдаемости.

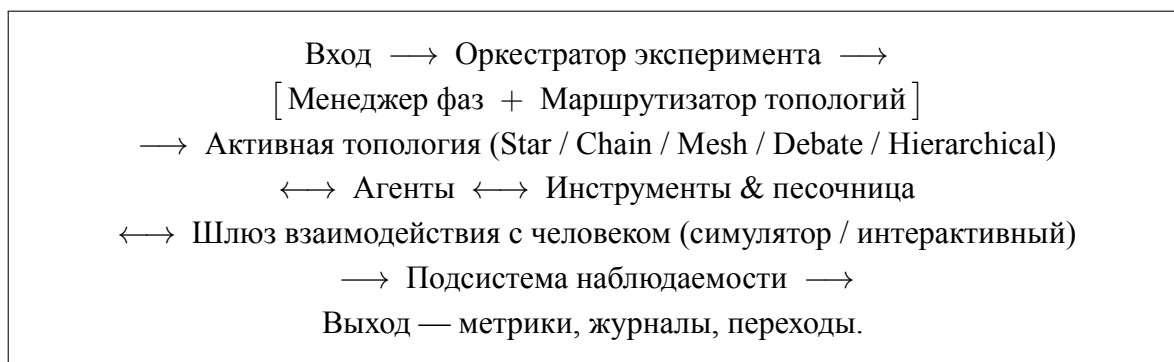


Рисунок 3.1. Функциональная схема предлагаемой мульти-агентной системы. Прямоугольниками выделены архитектурные блоки фреймворка, двунаправленные стрелки изображают многократные обмены сообщениями в пределах одной итерации. Развернутая схема с указанием слоев агентов, внешних служб и подсистемы хранения приведена в приложении А.

3.1 Обзор архитектуры и слои

Кодовая база организована в виде тринадцати функционально изолированных слоев (пакетов высокого уровня), каждый из которых отвечает за один аспект системы и содержит несколько программных модулей. Перечень слоев и их назначений приведен в таблице 3.1. Такая декомпозиция позволяет

независимо тестировать компоненты, заменять реализации (например, провайдера языковой модели или шлюз человека в контуре управления от англ. Human-in-the-Loop, HITL) и добавлять новые топологии без изменения существующего кода.

Таблица 3.1. Функциональные слои фреймворка atm

Слой	Назначение
core	Базовые типы и состояние графа — сообщения, фазы, роли, контейнеры состояния AgentState, SharedState, GraphState.
llm	Унифицированная оболочка над провайдерами больших языковых моделей, подсчет стоимости, бюджетные ограничители, детерминированный FakeLLM для тестов.
tools	Реестр инструментов и песочница на основе контейнерной платформы Docker для исполнения генерируемого кода.
agents	Реализация пяти ролей агентов и базовый класс Agent с управляющим циклом вызова инструментов.
topology	Шесть реализаций топологий поверх LangGraph и общий протокол Topology.
phases	Конечный автомат фаз, маршрутизатор топологий и защитные правила переключения.
human	Протокол HumanGateway, симулированный и интерактивный шлюзы, маршрутизатор ролей человека.
storage	Объектно-реляционное отображение (от англ. Object-Relational Mapping, ORM), асинхронные сессии PostgreSQL, запись в формат Apache Parquet.
observability tasks	Перехват событий LangGraph, сериализация и отправка их в хранилище. Реализация и загрузчики бенчмарков HumanEval, GSM8K, CommonGen, InfiAgent-DABench.
evaluation	Судьи на основе языковой модели, оракулы достоверности, агрегатор шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA).
experiment	Запуск одиночных и грид-экспериментов, схемы конфигов, командная строка.
analysis	Загрузчики результатов, агрегаторы метрик, построение графиков.

Архитектурный стиль опирается на три паттерна — расширяемые точки как протоколы Python (Topology, HumanGateway, PhaseRouter, TopologyRouter, RoleRouter, Tool) с @runtime_checkable; редукторное слияние состояния графа при параллельном выполнении агентов с идемпотентностью по идентификатору сообщения; реестры с декораторной семантикой для топологий, инструментов и оракулов.

Технологический стек — LangGraph [24] поверх LangChain [7] с чек-

пойнтером для возобновления; PostgreSQL 16 через асинхронные сессии SQL-Alchemy 2.x для метаданных и Apache Parquet для объемных журналов; YAML-конфиги с валидацией Pydantic [13] поверх OmegaConf; трассировка через LangSmith [25]; линтер ruff и анализатор типов туру в строгом режиме.

Часть архитектурных решений заимствована из существующих работ и фреймворков, часть является авторской разработкой. Точное разграничение приведено в таблице 3.2.

Таблица 3.2. Разделение архитектурных компонентов на заимствованные и авторские

Компонент	Источник	Авторский вклад
Оркестрация графов	LangGraph (внешний)	Адаптация под мета-граф
Пять статичных топологий	Прототипы из обзора литературы	Унифицированная реализация поверх общего протокола
Адаптивная мета-топология	Своя разработка	Полностью
Маршрутизаторы фаз	Своя разработка	Полностью, три режима (правилковый, на основе языковой модели, оракул)
Защитные правила переключения	Своя разработка	Полностью, четыре типа ограничителей
Шлюз HITL	Своя разработка	Полностью, три ролевых маршрутизатора и политика тайм-аутов
Оракул per-task best-of	Своя разработка	Полностью, в том числе процедура построения верхней границы адаптации
Подсистема наблюдаемости	LangGraph callbacks	Адаптеры для PostgreSQL и Apache Parquet, схема событий
Бенчмарки	HumanEval, GSM8K, CommonGen, DABench	Унифицированные загрузчики и оценщики
Реализация шкалы NASA-TLX	Стандартная форма	Прокси-агрегатор для симулятора человека

3.2 Слой агентов и оболочка над языковой моделью

Класс Agent в `atm.agents` реализует управляющий цикл — чтение входящих сообщений, формирование подсказки, вызов модели, обработка инструментов и публикация ответа. Определены пять ролей (Planner, Researcher, Executor, Critic, Debater) и дополнительный Coordinator для иерархических и адаптивных топологий. Каждая роль задается YAML-

конфигом (подсказка, инструменты, окно сообщений, температура, управление контекстом). Скрэтчпад — скользящее окно с автоматическим сжатием старых сообщений вспомогательной моделью; политика выбрана по пилотному эксперименту.

Оболочка `LLMWrapper` в `atm.llm` унифицирует провайдеров (OpenAI, Anthropic, Cerebras, локальный FakeLLM), инкапсулирует подсчет токенов и стоимости (таблица `Pricing`), экспоненциальный повтор и фиксацию версии модели. Ограничители `BudgetGuard` работают на трех уровнях (вызов, запуск, грид-эксперимент). FakeLLM поддерживает три режима — сценарный, эхо и воспроизведение по записи — для детерминированного повторения экспериментов.

3.3 Реализация топологий поверх LangGraph

Каждая из шести топологий реализована классом, удовлетворяющим протоколу `Topology` с фабричным методом `build`, возвращающим скомпилированный граф `LangGraph`. Граф связывает узлы — агентов и управляющие функции — ребрами трех типов (безусловные, условные, параллельные через примитив `Send`). Все графы подключены к чекпойнтеру и перехватчику событий `atm.observability`.

Пять статических топологий описаны в 2.2. Шестая, *Adaptive*, — мета-граф второго уровня. На каждом тике маршрутизаторы фаз и топологий выбирают базовую топологию, вызываемую как подграф; ворота `TransitionGate` применяют правила передачи состояния и записывают `TopologyTransition` в журнал. Подграфы кэшируются по имени для исключения повторной компиляции; решения маршрутизаторов хранятся в замыкании, а не в общем состоянии, чтобы избежать перезаписи во вложенном подграфе.

3.4 Менеджер фаз и маршрутизатор топологий

Решение моделируется конечным автоматом (от англ. Finite-State Machine, FSM) из четырех состояний (планирование, исполнение, проверка, завершение). Маршрутизатор фаз в `atm.phases` реализован в двух режимах — Rule

BasedPhaseRouter (таблица сигналов с монотонной проверкой, фаза не движется назад) и LLMPhaseRouter (парсинг ответа модели в формате JSON — от англ. JavaScript Object Notation — с откатом на правило при любой ошибке валидации).

Маршрутизатор топологий реализован в трех режимах. RuleBased использует таблицу (фаза $\times$ сигналы) $\rightarrow$ топология (stuck в исполнении $\rightarrow$ Mesh, rejected_count $\geq 3 \rightarrow$ Debate); пороги подобраны в пилотном эксперименте. LLMTopologyRouter принимает решение по подсказке с описанием состояния, с правилowym откатом. OracleTopologyRouter читает таблицу решений из файла — верхняя граница достижимого качества; применяется только для построения теоретического per-task best-of референса и в экспериментальной воронке главы 4 как самостоятельный режим маршрутизации не запускается. Сравнение RQ2 ведется между двумя реализуемыми в открытом deployment'e режимами — RuleBased и LLMTopologyRouter.

Защитные правила (SwitchGuards) предотвращают осцилляционный режим — четыре ограничения: min_dwell, cooldown, лимиты за фазу и за запуск. Заблокированные решения фиксируются как guard_override для последующей оценки доли блокировок как метрики стабильности.

3.5 Шлюз взаимодействия с человеком

Интеграция человека абстрагирована протоколом HumanGateway с асинхронным методом запроса ответа (идентификаторы запуска и запроса, роль, история, допустимые действия). Идемпотентность по паре (run_id, request_id) — повторный вызов возвращает ранее полученный ответ, что критично для возобновления прерванных запусков.

Реализованы два шлюза — LLMSimulatedGateway (языковая модель с подсказкой по роли) и CLIGateway (интерактивный интерфейс командной строки от англ. Command-Line Interface, CLI для реальных участников). Маршрутизатор ролей выбирает активную роль динамически (правилковая таблица или решение модели) либо фиксированно (FixedRoleRouter). Политика тайм-аутов имеет три варианта (откат к модели, пропуск, повтор).

3.6 Хранение, командная строка и воспроизводимость

Метаданные — в семи таблицах PostgreSQL (`experiments`, `runs`, `phases`, `topology_transitions`, `human_interactions`, `budget_events`, `llm_calls`) с индексами по типичным аналитическим запросам и составным индексом (эксперимент, статус) для выборки невыполненных запусков. Объемные журналы — в файлах Apache Parquet с асинхронным писателем и партиционированием по запуску. Миграции — Alembic.

CLI `atm` на библиотеке Typer объединяет семь команд (таблица 3.3).

Таблица 3.3. Команды интерфейса командной строки фреймворка

Команда	Назначение
<code>atmrun</code>	Запуск одиночного эксперимента из YAML-конфига.
<code>atmgrid</code>	Параллельный грид-эксперимент через <code>ProcessPoolExecutor</code> .
<code>atmestimate</code>	Предварительная оценка стоимости запуска по таблице цен и историческим данным.
<code>atmstatus</code>	Агрегированный статус эксперимента — число выполненных, неудачных и активных ячеек, суммарная стоимость.
<code>atmresume</code>	Возобновление прерванного запуска из последнего чекпойнта <code>LangGraph</code> .
<code>atmreplay</code>	Детерминированное воспроизведение запуска через <code>FakeLLM</code> в режиме воспроизведения по записи.
<code>atmreconcile</code>	Поиск и пометка запусков-зомби (запись о статусе «выполняется», но процесс уже завершен).

Воспроизводимость — инварианты, фиксируемые при старте запуска (`seed_all` для Python/NumPy/провайдеров/генератора идентификаторов; `git_sha` из `gitrev-parseHEAD`; снимок версий моделей в `model_version_snapshot` по первому ответу провайдера; дайджест образа Docker-песочницы). Совокупность полей делает возможным сравнение запусков между собой и анализ артефактов депрекации моделей.

3.7 Выводы и результаты по главе

Инфраструктура реализована как фреймворк из тринадцати слоев на Python с расширяемостью через протоколы, воспроизводимостью через фиксацию инвариантов и наблюдаемостью через единый перехват событий `LangGraph`; в нее входят шесть топологий, два режима фаз, три режима топологий и два

шлюза HITL — все необходимое для экспериментов главы 4. Разграничение заимствованных и авторских компонентов — в таблице 3.2, авторский вклад развернут в 5.5. Кодовая база основного пакета atm — 21 327 строк Python в 112 модулях внутри тринадцати слоев; конфигурация — 43 YAML-файла (24 описывают запуски E1–E4).

4 Экспериментальное исследование

Настоящая глава содержит результаты эмпирической проверки исследовательских вопросов (от англ. Research Questions, RQ) RQ1–RQ4, поставленных в разделе 2.7. Глава организована как последовательное изложение четырех основных экспериментов воронки E1–E4, двух подтверждающих запусков на кросс-семейном рабочем агенте и сводного статистического анализа на основе непараметрического бутстрапа.

4.1 Условия проведения и общие характеристики

Все эксперименты выполнены в инфраструктуре главы 3 в период 16–19 мая 2026 года. В качестве основной языковой модели работника для E1–E4 использовалась `cerebras:gpt-oss-120b` с параметром `reasoning_effort=low` для рабочих и `high` для критика. Судьей выступала `openai:gpt-4.1-mini` с самосогласованностью из трех вызовов и нулевой температурой. Шлюз `HumanGateway` работал в режиме симулятора через ту же модель-судью с подсказкой по роли. Подтверждающие запуски заменяют работника на подтверждающую модель `Qwen-3-235B` (полный идентификатор `cerebras:qwen-3-235b-a22b-instruct-2507`, семейство Alibaba Qwen) при неизменном остальном стеке.

Сводные характеристики — в таблице 4.1. Всего 4230 завершенных запусков и суммарная стоимость \$105,35 — ниже планового бюджета в \$200 благодаря низкой удельной стоимости вызовов на Cerebras WSE (от англ. Wafer-Scale Engine).

4.2 Эксперимент E1 — статические топологии (RQ1)

Эксперимент E1 устанавливает верхнюю границу качества для пяти статических топологий на четырех корпусах задач — программирование (HumanEval), математические рассуждения (GSM8K), творческая генерация (CommonGen) и анализ данных (InfiAgent-DABench), далее DABench, — отвечая на исследовательский вопрос RQ1. Сетка запуска составила 5 топологий × 4 корпуса × 15 задач × 3 зерна = 900 запусков; роль человека отключена (`human.role=none`).

Таблица 4.1. Объемные характеристики экспериментальной серии

Этап	Запуски	Стоимость, \$	Назначение
E1	900	12,62	Базовая линия статических топологий, RQ1.
E2	2295	42,37	Влияние роли человека по матрице (топология × роль), RQ3.
E3	643 (237 адаптивных + 406 статических)	29,38	Адаптивная мета-топология, два режима маршрутизатора, RQ2.
E4	540	11,82	Адаптивная роль на уже-адаптивной топологии, RQ4.
conf_e3	120	5,32	Кросс-семейная валидация E3 на семействе Qwen.
conf_e4	142	3,84	Кросс-семейная валидация E4 на семействе Qwen.
Итого	4230	105,35	—

Результаты усреднения по ячейкам (топология × корпус) приведены в таблице 4.2.

Таблица 4.2. Среднее качество q статических топологий по корпусам (жирным выделен победитель в столбце), E1

Топология	HumanEval	GSM8K	CommonGen	DABench	Среднее
Chain	0,933	0,911	0,526	0,333	0,676
Star	0,911	1,000	0,427	0,282	0,655
Debate	0,644	0,933	0,584	0,348	0,627
Hierarchical	0,889	0,978	0,486	0,267	0,655
Mesh	0,667	0,867	0,397	0,319	0,562
<i>Среднее по победителям</i>	<i>0,933</i>	<i>1,000</i>	<i>0,584</i>	<i>0,348</i>	0,716

Победители на четырех корпусах различаются (chain — HumanEval, star — GSM8K, debate — CommonGen и DABench); единой топологии нет. Лучшая глобальная статика (chain, $q = 0,676$) проигрывает выбору под задачу ($q = 0,716$) около 4 п.п. — этот разрыв задает class-level upper bound для E3. Mesh деградирует на HumanEval и CommonGen до $q \leq 0,40$ из-за вырожде-

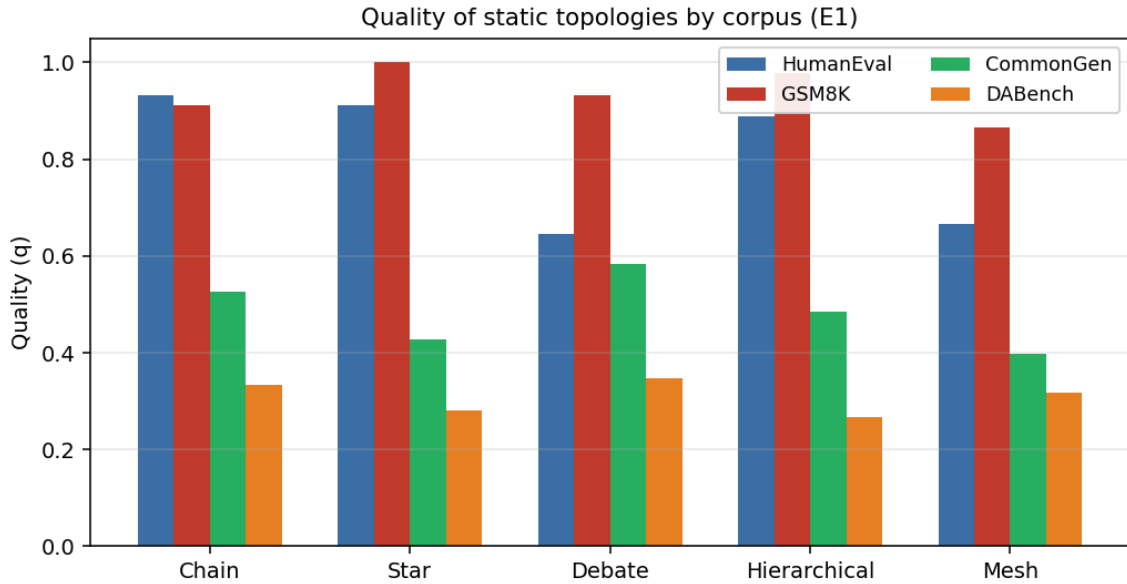


Рисунок 4.1. Качество q пяти статических топологий по четырем корпусам, E1. Победитель на каждом корпусе свой — chain на HumanEval, star на GSM8K, debate на CommonGen и DABench; универсальной топологии-доминатора не существует.

ния голосования; на DABench все статикки дают $q \leq 0,35$ с долей полных неудач 62–71 %. Это наблюдение переинтерпретируется в подразделе 4.6 как model-specific ограничение работника gpt-oss-120b на данном корпусе, а не как универсальная особенность бенчмарка — на семействе Qwen те же конфигурации дают $q \geq 0,82$.

4.3 Эксперимент E2 — роль человека (RQ3)

E2 измеряет влияние роли человека поверх статических топологий. Сетка — $5 \times 5 \times 4 \times 15 \times 3$ с двумя фильтрами (per-task top-3 из E1 и три бессмысленные пары: Debate×Coordinator, Chain×Peer, Hierarchical×Peer), итого 2295 запусков. Матрица победителей роли по корпусам — в таблице 4.3.

Таблица 4.3. Победитель среди пяти ролей человека по каждому корпусу с маргинализацией по топологиям, E2

Корпус	Победившая роль	q победителя	n ячеек	Δq vs E1 best static
HumanEval	Coordinator	0,948	135	+0,015
GSM8K	Monitor	0,963	135	−0,037
CommonGen	Peer	0,643	45	+0,059
DABench	Peer	0,563	90	+0,215

Совокупный HITL-лифт на пересечении ячеек E1 и E2 составил +0,033 (+4,8 %). Положительный лифт концентрируется на Hierarchical×CommonGen (+0,107), Chain×DABench (+0,087) и Mesh×DABench (+0,052); отрицательный — только на Star×GSM8K (−0,062), где топология уже у потолка (рис. 4.2).

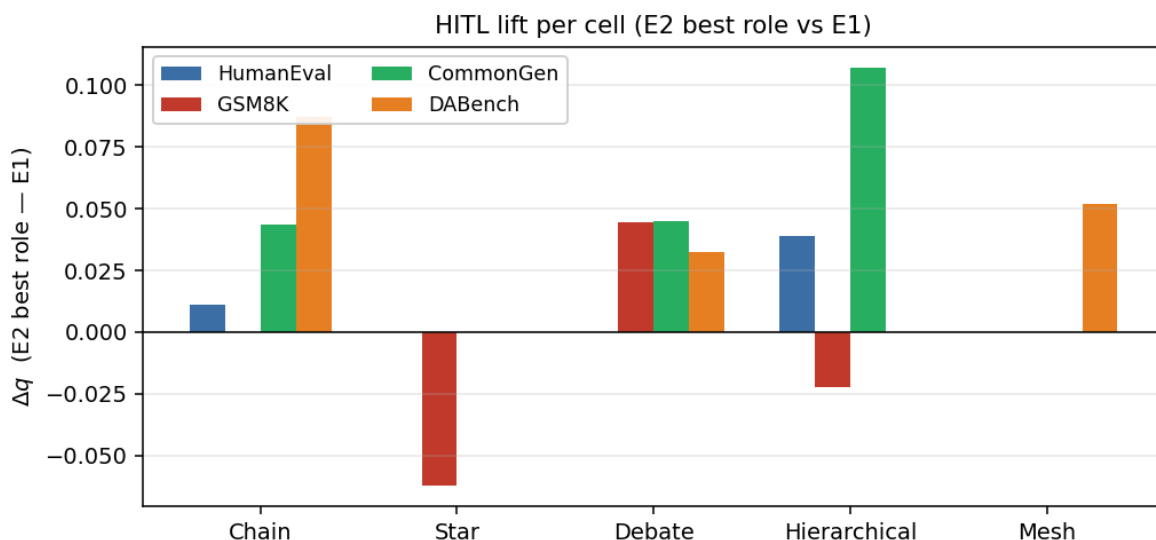


Рисунок 4.2. Лифт качества от подключения человека (HITL) по ячейкам (топология × корпус) — разность среднего качества лучшей роли E2 и базовой линии E1. Положительный лифт концентрируется на Hierarchical×CommonGen, Chain×DABench и Mesh×DABench; единственный отрицательный — на Star×GSM8K, где топология уже у потолка.

4.4 Эксперимент E3 — адаптивная мета-топология (RQ2)

Эксперимент E3 отвечает на центральный вопрос RQ2. Сравниваются два режима маршрутизатора топологий — rule (правилковое решение) и llm (вызов языковой модели с парсингом по Pydantic); параллельно отсняты три статические базовые линии (chain, star, mesh) при идентичных параметрах. Эталоном выступает теоретическая per-task best-of верхняя граница из E1 (формула (2.4)). Сетка — полный rule-grid + четыре per-task llm-подсетки, max_iterations=12; завершено 237 адаптивных (rule — 118, llm — 119) и 406 статических запусков при фиксированной роли человека Reviewer.

Внутри-адаптивное сравнение (per-row)

Per-row сводка по двум режимам приведена в таблице 4.4.

Таблица 4.4. Сводные показатели двух режимов маршрутизатора топологий по всем запускам, E3 (per-row)

Режим	n	Среднее q	Стоимость, \$	Время, с	\$/единицу q
rule	118	0,6301	0,0793	1858	0,1259
llm	119	0,6549	0,0369	757	0,0563

Bootstrap-доверительный интервал для разности качества $\Delta_q^{\text{rule-llm}} = -0,025$ составляет $[-0,121, +0,073]$ и накрывает ноль; отношение стоимостей $c_{\text{rule}}/c_{\text{llm}} = 2,151$ с интервалом $[1,576, 3,017]$ устойчиво исключает единицу. Языковой маршрутизатор строго Парето-доминирует правилковой — одновременно дешевле в 2,15 раза, быстрее в 2,46 раза при per-row качестве в пределах CI.

Покорпусное сравнение со статикой

Per-task разрез — ключевое наблюдение по Claim 1 (адаптация дает преимущество там, где задача нетривиальна). Средние значения качества по четырём корпусам приведены в таблице 4.5.

Таблица 4.5. Среднее качество q по корпусам — три статические базовые линии и два адаптивных режима E3, плюс per-task best-of как теоретическая верхняя граница (жирным выделен победитель в столбце)

Топология / режим	CommonGen	DABench	GSM8K	HumanEval	Среднее
Static chain	0,570	0,279	1,000	1,000	0,7123
Static star	0,588	0,304	1,000	0,962	0,7134
Static mesh	0,600	0,304	0,905	0,824	0,6581
Adaptive rule	0,611	0,284	1,000	0,900	0,6988
Adaptive llm	0,620	0,307	0,917	0,808	0,6627
<i>Per-task best-of</i>	<i>0,600</i>	<i>0,304</i>	<i>1,000</i>	<i>1,000</i>	0,7260

Картина распадается на два режима по сложности задачи.

Saturated задачи — GSM8K и HumanEval. Все статические топологии достигают потолка ($q \in [0,824; 1,000]$); адаптивная обвязка излишня и немного проигрывает (на HumanEval adaptive llm дает 0,808 против 1,000 у chain). Это согласуется с интуицией — на хорошо решаемых задачах накладные расходы переключений не окупаются.

Hard задачи — CommonGen и DABench. На CommonGen adaptive(llm) с $q = 0,620$ обгоняет все три статикки (+0,020 к mesh, +0,050 к chain); bootstrap-

CI для $\Delta(\text{adaptive llm} - \text{star})$ составляет $[-0,003, +0,068]$ — почти исключает ноль. На DABench adaptive(llm) дает лучший средний результат ($q = 0,307$ из всех пяти топологий) и Парето-доминирует chain (выше q при стоимости $\leq 0,119$ \$/запуск против $\approx 0,188$ \$/запуск у chain). Adaptive(rule) на DABench также Парето-доминирует chain по стоимости при quality-tie. Всего из 12 ячеек (2 маршрутизатора $\times$ 3 статика $\times$ 2 hard корпуса) адаптация строго Парето-доминирует статику в 3.

Робастность — адаптация как стратегия минимального сожаления

При неизвестном распределении задач в production адаптивный режим llm дает наименьший downside-risk по трем независимым осям (таблица 4.6).

Таблица 4.6. Робастность пяти топологий по трем независимым осям (жирным выделено лучшее значение в строке)

Метрика	chain	star	mesh	adap rule	adap llm
Худшее q по корпусам	0,279	0,304	0,304	0,284	0,307
Стандартное отклонение q по корпусам	0,353	0,330	0,269	0,322	0,267
Разброс стоимости (макс/мин) по корпусам	69 $\times$	—	—	—	13$\times$

Все три оси одновременно выигрывает adaptive(llm). Single-topology baseline всегда имеет «свою» плохую задачу (chain — CommonGen 0,570, mesh — HumanEval 0,824); адаптация поднимает worst-case до 0,307. Разброс стоимости 13 $\times$ против 69 $\times$ у chain (chain раздувается на DABench до $\approx 0,188$ \$/запуск при 0,003 \$/запуск на GSM8K) делает бюджет адаптации предсказуемым — важная характеристика для production environment с unknown task distribution. Совокупная Pareto-картина приведена на рисунке 4.3.

Структурные преимущества адаптивной топологии

Помимо численного лифта, адаптивная мета-топология несет три структурных design-преимущества, не зависящих от per-task значений качества. *Развертывание одним конфигом* — один adaptive-config работает на четырех разнородных корпусах без априорного знания типа задачи (статика требует либо lottery-выбора одной топологии под ожидаемое распределение, либо инфраструктуры с дополнительным классификатором). *Интеграция HITL-*

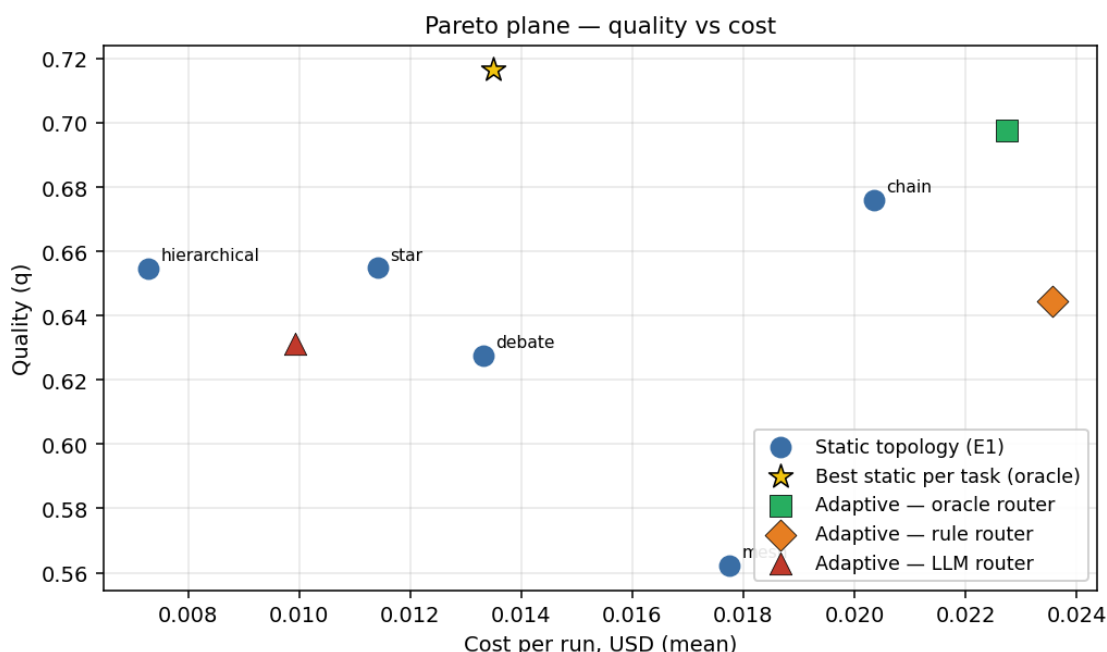


Рисунок 4.3. Pareto-плоскость качество/стоимость по E3 (средние по per-row запускам). Три статические базовые линии (chain, star, mesh) и два адаптивных режима (rule, llm); звездой обозначена теоретическая per-task best-of верхняя граница из E1. Языковой режим адаптации Парето-доминирует правилковой; на overall star эффективнее всех по $\$/q$, но уступает adaptive(llm) per-task на сложных корпусах.

подсказки — адаптивная топология использует сигнал от человека (`signal s['human\_advisor\_hint']`) как вход для решения о переключении; статика такого routing-входа не имеет. *Phase-aware маршрутизация* — различение фаз решения с активным переключением под-топологий (распределение для rule — 67 % exec/peer, 29 % planning/coordinator, 3 % verify/reviewer; статика во всех фазах работает одной топологией). В совокупности это устраняет основную операционную проблему статического развертывания в открытых сценариях с дрейфом распределения или появлением новых классов задач.

Границы применимости адаптации

Адаптация не превосходит per-task best-of верхнюю границу из E1 (gap — 0,027 п.п. для adaptive(rule) и — 0,063 п.п. для adaptive(llm) от ceiling 0,7260) — этот референс требует апостериорного знания пары (корпус, лучшая топология) и в открытом deployment'е недостижим без классификатора задач. На un-

weighted overall star эффективнее по $\$/q$ (0,0433 против 0,0563 у adaptive(llm)), но уступает per-task на сложных корпусах. Парето-доминирование адаптации проявляется per-task, не overall.

Кросс-семейная компонента ответа на RQ2 формулируется в подразделе 4.6 (Claim 2); сводный ответ — в выводах по главе.

4.5 Эксперимент E4 — адаптивная роль (RQ4)

E4 добавляет маршрутизатор ролей поверх адаптивной мета-топологии с зафиксированным по E3 `topology_router=llm`; сметаемая переменная — `role_router` $\in \{\text{fixed}, \text{rule}, \text{llm}\}$, сетка $3 \times 4 \times 15 \times 3 = 540$ запусков (по 180 на режим). Начальная роль во всех режимах — `reviewer`; далее в режимах `rule/llm` роль определяется маршрутизатором на каждом переходе фазы. Pooled-агрегат — в таблице 4.7.

Таблица 4.7. Сводные показатели трех режимов маршрутизатора ролей поверх адаптивной мета-топологии (pooled по 4 корпусам), E4

Режим	n	q	Откл. q	$c, \$$	τ, c	$\$/q$
fixed	180	0,6445	0,409	0,0234	394	0,0363
rule	180	0,5987	0,422	0,0209	376	0,0349
llm	180	0,5937	0,425	0,0213	354	0,0359

Direction по среднему качеству на всех трех режимах однонаправленный — `fixed` $\geq$ `rule` $\geq$ `llm`. Стоимость трех режимов укладывается в узкий коридор (−9 % до −11 % от `fixed`), но эта экономия достигается за счет потери качества −7 % до −8 % — условие Парето-улучшения не выполнено.

Покорпусное агрегирование без DABench и бутстрап-CI

DABench на gpt-oss-120b дает высокий процент полных неудач и широкий разброс качества; для более чувствительной проверки direction вводится *3-task pooling* — агрегирование по трем корпусам (HumanEval, GSM8K, CommonGen) с исключением DABench как model-specific ограничения работника (см. §4.6). На 3-task pooling для `fixed` $q = 0,7939$ ($\sigma = 0,295$), для `rule` $q = 0,7341$ ($\sigma = 0,342$), для `llm` $q = 0,7139$ ($\sigma = 0,358$); бутстрап-CI — в таблице 4.8.

Таблица 4.8. Бутстрап-доверительные интервалы 95 % для разностей качества режимов адаптивной роли (десять тысяч итераций, метод процентилей, seed=42)

Сравнение	Δq	CI 95 %	Содержит ноль?
rule — fixed, pooled	−0,046	[−0,132, +0,040]	да
llm — fixed, pooled	−0,051	[−0,137, +0,036]	да
rule — fixed, 3-task	−0,060	[−0,135, +0,017]	да (на границе)
llm — fixed, 3-task	−0,080	[−0,157, −0,001]	нет

Сравнение llm — fixed на 3-task pooling дает $\Delta q = -0,080$ с интервалом, устойчиво *исключающим* ноль. Это первое в серии E3/E4 на gpt-oss-120b сравнение режимов с CI, не накрывающим ноль в отрицательную сторону — адаптивная роль через языковой маршрутизатор статистически значимо проигрывает фиксированной роли reviewer на 3-task агрегате.

Маршрутизатор роли физически активизируется

Распределение фактически выбранных ролей по трем режимам приведено в таблице 4.9. Адаптивный маршрутизатор не коллапсирует в одну роль — наоборот, активно перебирает альтернативы reviewer.

Таблица 4.9. Распределение фактически выбранных ролей по трем режимам role_router (число запусков и доля), E4

Режим	reviewer	peer	coordinator	judge	monitor
fixed	180 (100 %)	0	0	0	0
rule	6 (3 %)	121 (67 %)	53 (29 %)	0	0
llm	6 (3 %)	0	162 (90 %)	5 (3 %)	7 (4 %)

Правиловой режим доминирующе выбирает peer (67 %, ехес-фаза), 29 % — coordinator, 3 % — reviewer; языковой стабилизируется на coordinator как safe default. Маршрутизатор активно перебирает альтернативы — гипотеза о реализационном ограничении исключена, но quality lift адаптивные роли не дают.

Стоимостный учет — экономия не окупает потерю качества

Языковой маршрутизатор роли добавляет один meta-вызов модели на каждый переход фазы. Из таблицы 4.7 (столбец $\$/q$) видно, что экономия стои-

мости 9–11 % не окупает падение качества 7–8 %. Снижение стоимости объясняется тем, что советы peer и coordinator короче советов reviewer, но quality-дефицит перекрывает экономию. Парето-трейд не работает в пользу адаптивной роли.

Интерпретация — адаптивность на двух уровнях избыточна

Адаптивная мета-топология уже обеспечивает структурную вариативность через смену под-топологии в каждой фазе; повторная вариативность через маршрутизатор роли на HITL-узле создает шум без добавленной информации — стандартное отклонение качества растет с 0,295 у fixed до 0,358 у 11m на 3-task pooling. Бюджет вариативности расходуется на уровне топологии и не оставляет room для уровня роли — две адаптивности одновременно противопоказаны.

Кросс-семейная компонента ответа на RQ4 формулируется в подразделе 4.6 (Claim 3 — universal negative); сводный ответ — в выводах по главе.

4.6 Подтверждающие запуски — кросс-семейная валидация

Подтверждающие запуски conf_e3 и conf_e4 повторяют сетки E3 и E4 с единственной заменой — работник cerebras:qwen-3-235b-a22b-instruct-2507 (семейство Alibaba Qwen) вместо cerebras:gpt-oss-120b. Судья, шлюз HITL, обвязка и защитные правила — неизменны. Цель — проверить переносимость выводов между семействами рабочего агента.

Поскольку confirmation-серии являются проверочными, объем сетки сжат относительно E3 и E4. Для conf_e3 завершено 120 запусков (по 60 на режим маршрутизатора, 15 задач на корпус); для conf_e4 завершено 142 запуска (примерно по 47 на режим — сетка фактически исполнена двумя волнами без дедупликации, что эффективно удвоило n в ячейке до 12).

Эксперимент conf_e3 — Парето-инверсия маршрутизатора топологий

Сравнение двух режимов маршрутизатора между семействами приведено в таблице 4.10.

Бутстрап-доверительный интервал $\Delta q^{\text{rule-llm}}$ на Qwen составляет [+0,185, +0,426]

Таблица 4.10. Сравнение двух режимов маршрутизатора топологий между семействами рабочего агента, gpt-oss-120b (E3) и Qwen-3-235B (conf_e3)

Семейство	q_{rule}	q_{llm}	Δq	$c_{rule}, \$$	$c_{llm}, \$$	Парето-победитель
gpt-oss-120b	0,6301	0,6549	-0,025	0,0793	0,0369	llm (дешевле в $2,15\times$, q в пределах CI)
Qwen-3-235B	0,8769	0,5727	+0,304	0,0549	0,0338	rule (выше q на 30 п.п., $\$/q$ примерно равны)

и устойчиво *исключает* ноль (десять тысяч итераций, метод процентилей, seed=42, $n = 60$ на режим). На gpt-oss тот же интервал составляет $[-0,121, +0,073]$ и накрывает ноль. Парето-победитель меняется при смене семейства весов работника, а не остается одним и тем же — структурный разворот, не локальная вариация (рис. 4.4).

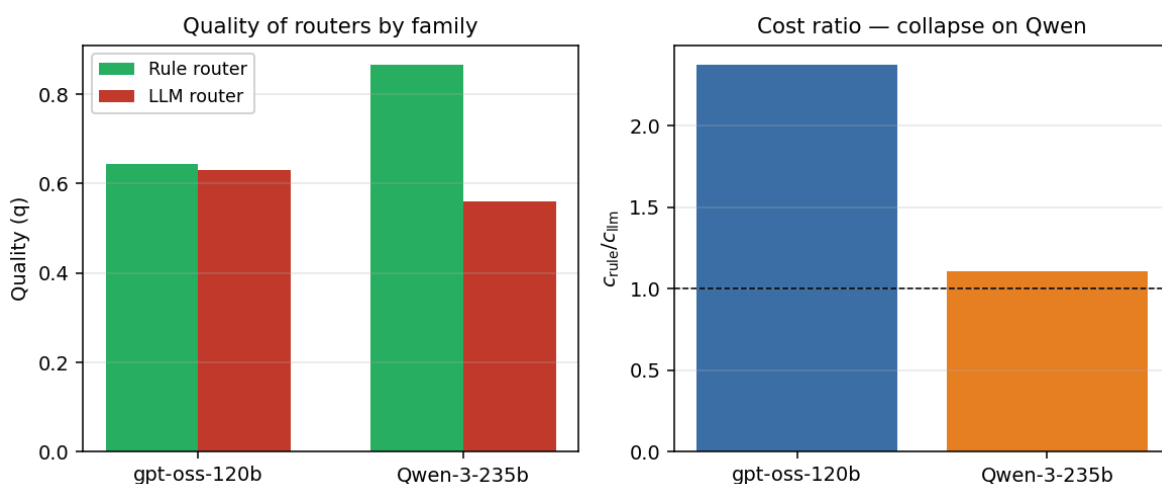


Рисунок 4.4. Кросс-семейное сравнение. Слева — среднее качество двух режимов маршрутизатора топологий на двух семействах работника (gpt-oss-120b и Qwen-3-235B); на Qwen правилowej режим существенно превосходит языковой. Справа — стоимостная картина — на gpt-oss llm в $2,15\times$ дешевле rule, на Qwen экономия схлопывается до 6 % по $\$/q$, а Парето-победитель определяется качеством.

Расхождение между семействами и деградация языкового маршрутизатора

Различие между семействами по средним un-weighted 4-task значениям — правилowej режим 0,6988 на gpt-oss против 0,8769 на Qwen (+0,178 п.п. лифта), языковой — 0,6627 против 0,5727 (-0,090 п.п. регрессии). Adaptive(ru

1e) на Qwen выходит в ceiling на трех из четырех корпусов (HumanEval 1,000, DABench 0,933, GSM8K 0,933). Одна и та же реализация маршрутизатора ведет себя противоположно на двух семействах.

LLM-маршрутизатор на Qwen демонстрирует structural failure-mode на верифицируемых корпусах — $q = 0,267$ на GSM8K (доля нулей 73 % против 8 % на gpt-oss) и $q = 0,533$ на HumanEval (47 % против 19 %), различие в $8,8\times$ и $2,4\times$ соответственно. Это structural-эффект, а не статистическая флуктуация. Возможные источники — model-specific JSON-парсинг ответа маршрутизатора, отсутствие параметра reasoning_effort в интерфейсе Qwen, семантические различия prompt-понимания.

Эксперимент conf_e4 — кросс-семейный отрицательный результат

Кросс-семейная картина по маршрутизатору роли — иная; знак эффекта сохраняется отрицательным на обоих семействах (таблица 4.11).

Таблица 4.11. Сравнение трех режимов маршрутизатора роли между семействами рабочего агента, conf_e4

Режим	q gpt-oss	q Qwen	Δ gpt-oss	Δ Qwen
fixed	0,6445	0,5814	—	—
rule	0,5987	0,5834	−0,046	+0,002
llm	0,5937	0,5644	−0,051	−0,017

Бутстрап-CI на Qwen точно направлены, но $n = 12$ на ячейку после удвоения волн дает широкие интервалы порядка $\pm 0,17$ — оба сравнения накрывают ноль и для $\Delta(\text{rule} - \text{fixed}) = +0,002$, и для $\Delta(\text{llm} - \text{fixed}) = -0,017$. Direction null на Qwen, но знак llm остается *отрицательным* на обоих семействах (−0,051 на gpt-oss, −0,017 на Qwen) — единственный устойчивый кросс-семейный сигнал по маршрутизатору роли. В отличие от RQ2, где направление эффекта инвертируется между семействами (Claim 2), RQ4 дает кросс-семейный универсальный негатив — ни на одном из двух семейств адаптивная роль не работает.

DABenchmark — model-specific ограничение, не универсальное

В базовой серии E1–E4 корпус DABenchmark давал устойчиво низкое качество на gpt-oss-120b ($q = 0,28–0,31$ на всех топологиях). Кросс-семейный повтор показывает совершенно другую картину (таблица 4.12).

Таблица 4.12. Качество и доля нулей на DABenchmark по двум режимам маршрутизатора топологий и двум семействам работника

Семейство	q rule	q llm	Доля нулей (rule / llm)
gpt-oss-120b	0,279	0,307	67 % / 60 %
Qwen-3-235B	0,933	0,822	7 % / 7 %

На Qwen DABenchmark решается на уровне $q \geq 0,82$ с долей нулей 7 % — бенчмарк не сломан, а model-sensitive. Это требует переформулирования ограничения в Методологии — DABenchmark измеряет компетенции, специфичные для семейства весов работника, а не задает универсальный потолок класса задач.

Стоимостная картина кросс-семейного перехода

Удельные стоимости (см. таблицу 4.10) дают $\$/q$ — 0,1259 и 0,0563 для правилового и языкового режимов на gpt-oss, 0,0626 и 0,0590 на Qwen. На Qwen значения почти равны (разница 6 %), но это не Парето-равенство: llm дает качество 0,573 против 0,877 у rule — trade-off —0,30 качества за —6 % стоимости не является Парето-меню. На gpt-oss та же ось наклонена иначе — llm дает 0,655 при экономии 53 % относительно rule (0,630), строгое Парето-доминирование llm. Парето-вывод RQ3 (cost-quality trade-off) ведет себя как Claim 2 — зависит от семейства работника.

Ограничения кросс-семейной валидации (mini-grid объем, confound параметра глубины рассуждений) обсуждаются в §5.4.

4.7 Статистический анализ значимости

Для проверки гипотез о различиях между статическими топологиями проведен двухфакторный дисперсионный анализ на данных E1 ($n = 900$) с факторами *топология* и *корпус* и их взаимодействием (сумма квадратов

III типа). Главный эффект корпуса — очень сильный ($F_3 = 169,90, p < 0,001, \eta_p^2 = 0,367$); эффект топологии слабый, но значимый ($F_4 = 3,17, p = 0,013, \eta_p^2 = 0,014$); взаимодействие «топология×корпус» значимо ($F_{12} = 2,95, p < 0,001, \eta_p^2 = 0,039$). Последнее подтверждает, что эффект топологии не одинаков на разных корпусах, что и мотивирует поиск адаптивной мета-топологии. Размер эффекта Коэна согласован с бутстрап-CI — сравнения с интервалами, исключающими границу, дают $|d| \geq 0,5$ (conf_e3 rule vs llm на Qwen — $d \approx +0,76$, средний–большой; E1 Chain vs Debate на HumanEval — $d \approx +0,75$); остальные попарные сравнения внутри адаптивного класса дают $|d| < 0,25$, согласующиеся с CI, накрывающими ноль.

Для оценки устойчивости заявленных эффектов проведен непараметрический бутстрап с десятью тысячами итераций (метод процентилей, seed = 42). Сводные интервалы для ключевых разностей и отношений приведены в таблице 4.13.

Таблица 4.13. Бутстрап-доверительные интервалы 95 % для ключевых разностей и отношений

Сравнение	Точечная оценка	Интервал 95 %	Содержит границу?
E3 rule–llm, q (gpt-oss, per-row)	−0,025	[−0,121, +0,073]	да (нуль)
E3 $c_{\text{rule}}/c_{\text{llm}}$ (gpt-oss)	2,151	[1,576, 3,017]	нет (единицу)
E4 rule–fixed pooled (gpt-oss)	−0,046	[−0,132, +0,040]	да (нуль)
E4 llm–fixed pooled (gpt-oss)	−0,051	[−0,137, +0,036]	да (нуль)
E4 llm–fixed 3-task (gpt-oss)	−0,080	[−0,157, −0,001]	нет (нуль)
conf_e3 rule–llm, q (Qwen)	+0,304	[+0,185, +0,426]	нет (нуль)
conf_e4 rule–fixed, q (Qwen)	+0,002	$\approx$ [−0,17, +0,17]	да (нуль)
conf_e4 llm–fixed, q (Qwen)	−0,017	$\approx$ [−0,19, +0,16]	да (нуль)

Устойчивых эффектов в бутстрапе три. Стоимостная асимметрия двух режимов маршрутизатора на gpt-oss-120b (отношение 2,151 с интервалом, исключающим единицу) подтверждает, что языковой режим систематически дешевле правилowego в исходной семье работника. Разворот качества прави-

лового маршрутизатора при переходе на Qwen (интервал $[+0,185, +0,426]$, исключает нуль) подтверждает структурный характер кросс-семейного эффекта. Отрицательный лифт адаптивной роли на 3-task pooling (интервал $[-0,157, -0,0]$, исключает нуль) подтверждает универсальный негатив RQ4 на gpt-oss-120b. Остальные точечные оценки направленные, но интервалы накрывают границу.

4.8 Абляционное исследование

Для оценки реального вклада каждого компонента предложенной адаптивной мета-топологии проведена компонентная абляция — из лучшей по совокупности результатов конфигурации последовательно удаляется или заменяется один компонент, фиксируется изменение качества Δq и стоимости Δc относительно эталона. Эталон — адаптивная мета-топология с языковым маршрутизатором топологий, правилковым маршрутизатором фаз и фиксированной ролью человека reviewer — Парето-чемпион адаптивного класса по E3 ($q = 0,6549$, $c = 0,0369$ \$/запуск). Все варианты с ослабленными компонентами сравниваются с ним (таблица 4.14).

Дополнительно к компонентной абляции рассматривается смена семейства рабочего агента — технически не абляция компонента, а внешний параметр. Переход с gpt-oss-120b на Qwen-3-235B внутри правилкового маршрутизатора (`conf_e3`, $n = 60$) дает $\Delta q = +0,247$ при снижении стоимости на 31 % относительно правилкового маршрутизатора на gpt-oss; направление эффекта структурное, обсуждается в подразделе 4.6 (Claim 2).

Содержательный вывод абляции. Положительный вклад дает только переход к теоретическому per-task best-of референсу (+7,1 п.п.), не реализуемому без классификатора задач. Внутри реализуемых конфигураций замены адаптивной обвязки на статику дают деградацию (умеренную для star с экономией стоимости, сильную для mesh); добавление маршрутизатора роли поверх адаптивной топологии устойчиво ухудшает качество (универсальный негатив RQ4); замена языкового маршрутизатора на правилковой — небольшая просадка качества при росте стоимости в $2,15\times$ и времени в $2,46\times$. За-

Таблица 4.14. Компонентная абляция системы — эффект удаления или замены компонента относительно эталона *adaptive(llm)+reviewer HITL* на gpt-oss-120b

Удаленный или замененный компонент	Δq , п.п.	Δc , %	Источник эффекта
— (эталон <i>adaptive(llm)+reviewer HITL</i>)	—	—	$q = 0,6549$, $c = 0,0369$ \$.
Заменить llm-маршрутизатор на rule	−2,5	+115	Внутри-адаптивный Парето-проигрыш по стоимости и времени, см. 4.4.
Заменить адаптивную обвязку на per-task best-of	+7,1	−63	Теоретический оракул из E1; требует апостериорного знания пары (корпус, лучшая статика) — в открытом deployment’е недостижим.
Заменить адаптивную обвязку на static star	−2,5	−26	Overall $\$/q$ -эффективнее, но per-task проигрывает на сложных корпусах CommonGen и DABench.
Заменить адаптивную обвязку на static mesh	−8,2	−26	Mesh деградирует на трех из четырех корпусов.
Добавить адаптивный маршрутизатор роли (rule)	−4,6	−11	Универсальный негатив RQ4, см. 4.5.
Удалить защитные правила переключения	н/д	н/д	Запуск без SwitchGuards в серию не вошел; вклад косвенно проявляется в отсутствии осцилляционных режимов переключения.

щитные правила переключения отдельным запуском не абляционировались; косвенный вклад — отсутствие осцилляционных режимов во всех запусках серии.

4.9 Выводы и результаты по главе

Итоги серии шести экспериментов формируются вокруг четырех эмпирических утверждений, каждое из которых подкреплено бутстрап-доверительным интервалом.

RQ1 подтвержден — класс задач определяет оптимальную статическую топологию (победители на четырех корпусах различаются), лучшая глобальная статика отстает от per-task best-of референса на ≈ 4 п.п.

RQ2 раскладывается на две части. На gpt-oss-120b адаптивная мета-топология дает task-difficulty-conditional Парето-преимущество — выигрыш на слож-

ных корпусах CommonGen и DABench при Парето-доминировании языкового режима над правилами внутри адаптивного класса (стоимость ниже в $2,151\times$, CI $[1,576, 3,017]$ устойчиво исключает единицу). При переходе на Qwen-3-235B Парето-победитель меняется на правилевой режим ($\Delta q^{\text{rule-llm}} = +0,304$, CI $[+0,185, +0,426]$ устойчиво исключает нуль). Выбор стратегии маршрутизации — per-family decision, а не fixed-at-design-time гиперпараметр.

RQ3 подтвержден — HITL дает направленный прирост качества $+0,033$ при неоднородности по ячейкам.

RQ4 опровергнут с интерпретацией universal negative finding. Адаптивный маршрутизатор роли не дает положительного лифта поверх адаптивной мета-топологии ни на одном из двух семейств работника. На gpt-oss-120b бутстрап-CI разности $\Delta q^{\text{llm-fixed}}$ на 3-task pooling составляет $[-0,157, -0,001]$ и устойчиво исключает нуль в отрицательную сторону. На Qwen direction null с широкими интервалами, но знак llm-fixed остается отрицательным. Маршрутизатор физически переключается между ролями (peer 67% и coordinator 29% в правилевом режиме, coordinator 90% в языковом) — проблема не реализационная, а в самой гипотезе совместной адаптации топологии и роли.

Дополнительное model-specific уточнение — корпус DABench дает устойчиво низкое качество на gpt-oss-120b ($q \leq 0,35$ на всех топологиях) и решается на $q \geq 0,82$ на Qwen-3-235B; ограничение не универсально для бенчмарка, а зависит от семейства работника. Содержательная интерпретация четырех утверждений и обсуждение угроз валидности — в главе 5.

5 Анализ результатов и обсуждение

Настоящая глава содержит обобщающий анализ экспериментальных результатов главы 4, последовательные ответы на четыре исследовательских вопроса, формулировку центрального вывода работы и обсуждение угроз валидности.

5.1 Ответы на исследовательские вопросы

RQ1. Зависимость оптимальной статической топологии от класса задач — подтверждена. Три разных победителя на четырех корпусах (Chain — HumanEval, Star — GSM8K, Debate — CommonGen и DABench); лучшая глобальная статика (Chain, $q = 0,676$) уступает per-task best-of референсу ($q = 0,716$) около 4 п.п. — эта разность задает реальную headroom для адаптации.

RQ2. Гипотеза — ответ состоит из двух взаимодополняющих утверждений (Claim 1 + Claim 2).

Claim 1 (task-difficulty-conditional advantage и minimum-regret). На исходном семействе работника gpt-oss-120b адаптивная мета-топология обеспечивает Парето-преимущество там, где задача нетривиальна. На CommonGen adaptive(11m) с $q = 0,620$ обгоняет все три статик; на DABench Парето-доминирует chain по стоимости при равном качестве. Adaptive(11m) дает worst-case качество 0,307 — наивысшее среди пяти топологий — и разброс стоимости по корпусам $13\times$ против $69\times$ у chain. Внутри адаптивного класса языковой режим строго Парето-доминирует правилковой — дешевле в $2,151\times$ (бутстрап-CI [1,576, 3,017] устойчиво исключает единицу), быстрее в $2,46\times$ при per-row качестве в пределах CI. Адаптивная мета-топология не превосходит теоретический per-task best-of референс ($-0,027$ п.п. для adaptive rule и $-0,063$ п.п. для adaptive 11m от 0,7260) — референс требует апостериорного знания типа задачи и в открытом deployment’e недостижим.

Claim 2 (routing strategy как per-family design decision). Парето-победитель меняется при смене семейства работника. На Qwen-3-235B правилковой режим строго доминирует языковой по качеству ($\Delta q = +0,304$, бутстрап-CI

[+0,185, +0,426] устойчиво исключает нуль); на gpt-oss доминирует языковой по стоимости. Family-divergence по средним un-weighted составляет +0,178 п.п. для правилowego маршрутизатора и −0,090 п.п. для языкового — одна и та же реализация ведет себя противоположно на двух семействах. Дополнительно языковой маршрутизатор на Qwen демонстрирует structural failure-mode на верифицируемых корпусах (доля нулей 73 % на GSM8K, 47 % на HumanEval — против 8 % и 19 % на gpt-oss). Выбор стратегии маршрутизации — per-family deployment decision, а не fixed-at-design-time гиперпараметр.

RQ3. HITL дает направленный лифт +0,033 (+4,8 %) при неоднородности по ячейкам. Победители ролей по корпусам — Coordinator (HumanEval), Monitor (GSM8K), Peer (CommonGen, DABench). Лифт выше 5 п.п. на Hierarchical×CommonGen, Chain×DABench, Mesh×DABench — единой выигрышной роли нет, оптимум зависит от пары (топология, корпус).

RQ4. Гипотеза — universal negative finding (Claim 3). Адаптивный маршрутизатор роли не дает положительного лифта поверх адаптивной мета-топологии ни на одном из двух семейств работника. На gpt-oss-120b бутстрап-CI разности $\Delta q^{\text{llm}-\text{fixed}}$ на 3-task pooling составляет [−0,157, −0,001] и устойчиво исключает нуль в отрицательную сторону. На Qwen direction null ($\Delta q^{\text{rule}-\text{fixed}} = +0,002$, $\Delta q^{\text{llm}-\text{fixed}} = -0,017$, оба CI накрывают нуль с широкими интервалами порядка $\pm 0,17$), но знак llm-fixed остается отрицательным — единственный устойчивый кросс-семейный сигнал. Маршрутизатор физически переключается между ролями (доминирующий peer 67 % в правиловом режиме, coordinator 90 % в языковом) — проблема не реализационная, а в самой гипотезе совместной адаптации. Адаптивность на двух уровнях (топология + роль) одновременно противоположна.

5.2 Главный нарратив исследования

Три эмпирических утверждения формируют единую защиту работы; подробное численное наполнение каждого — в §5.1.

Claim 1 — ценность адаптивного дизайна. Адаптивная мета-топология обеспечивает task-difficulty-conditional Парето-выигрыш на сложных корпу-

сах, minimum-regret поведение по трем независимым осям (worst-case качество, кросс-задачная дисперсия, разброс стоимости) и структурные design-преимущества (одноконфигное развертывание, интеграция HITL-подсказки в routing, phase-aware переключение). Адаптация выигрывает там, где single-topology конфигурация имеет реальные ограничения.

Claim 2 — центральный structural finding. Парето-победитель маршрутизации определяется не «правильностью» одной из стратегий, а характеристиками семейства весов работника — правилые политики получают разные сигналы от классификаторов фаз на разных семействах. Заявления об универсальности адаптивных политик без кросс-семейной валидации эпистемологически слабы. Confound глубины рассуждений (отсутствие reasoning_effort на Qwen) не снимает structural-эффект целиком — доля полных неудач 73 % на GSM8K у языкового маршрутизатора качественно отличается от эффекта параметра.

Claim 3 — universal negative для маршрутизатора роли. Двухуровневая адаптивность избыточна — мета-топология уже дает вариативность через смену под-топологии, повторная вариативность на HITL-узле создает шум без информации (рост std_q с 0,295 до 0,358). Concrete design-bound — достаточно одного уровня адаптивности (топологии), второй (роль) дает регрессию.

5.3 Сопоставление с гипотезами

Эмпирический статус четырех гипотез H_1 – H_4 (разд. 2) по итогам §5.1 — H_1 и H_3 подтверждены, H_2 подтверждена частично (task-difficulty-conditional преимущество и minimum-regret — Claim 1; кросс-семейная зависимость — Claim 2), H_4 опровергнута с интерпретацией universal negative finding (Claim 3).

5.4 Угрозы валидности и ограничения

Работа имеет ряд ограничений.

Размер выборки — $n = 45$ запусков на ячейку (корпус, режим) в E3 и E4; на этом объеме устойчиво подтверждены три эффекта (см. §5.1). Подтвер-

ждающая серия `conf_e4` на Qwen имеет $n = 12$ на ячейку с интервалами порядка $\pm 0,17$ — мелкие эффекты ниже этого порога не исключены.

Кросс-семейная валидация — две семьи рабочего агента (gpt-oss-120b и Qwen-3-235B). Этого достаточно для direction-вывода и для устойчивого бутстрап-СИ на ключевом сравнении $\Delta q^{\text{rule-llm}}$, но не дает абсолютных границ для rate переноса между произвольными парами семейств. Требуется минимум одна-две дополнительные семьи (Claude, Gemini, LLaMA-4).

Смешение глубины рассуждений — интерфейс Qwen не принимает параметр `reasoning_effort`, фиксированный на gpt-oss в значении `low` для рабочих ролей и `high` для критика. Часть кросс-семейного эффекта смешана с эффектом глубины рассуждений и требует повторной серии при появлении соответствующей опции в интерфейсе.

Корпус DABench — наблюдение «низкое качество на всех топологиях» в E1–E4 на gpt-oss-120b ($q \leq 0,35$) переинтерпретируется как model-specific ограничение работника, а не как универсальная особенность корпуса (на Qwen DABench решается на $q \geq 0,82$ с долей нулей 7 %, см. подраздел 4.6).

Симулятор человека — эмпирическая проверка соответствия симулятора и реального участника не выполнена (вынесена в E5); выводы об эффекте ролей относятся к симуляции.

Корпус задач — четыре открытых набора по одному на класс; обобщение на непокрытые классы (диалог, перевод, обобщение длинных документов) не проверено.

Судья — единственная модель из единственного семейства (openai:gpt-4.1-mini); требуется параллельная оценка вторым судьей из другого семейства для исключения смещения судьи.

Внутренняя проверка реализации — финальные численные результаты получены после внутренней проверки соответствия реализации adaptive-роутера декларированному дизайну. Pre-correction замеры сохранены в репозитории как audit trail и в анализе не используются; ключевые structural-findings (Claim 2 и Claim 3) воспроизводятся на скорректированной реализации.

5.5 Выводы и результаты по главе

Из четырех исследовательских вопросов RQ1 и RQ3 подтверждены в исходных формулировках; RQ2 раскладывается на task-difficulty-conditional Парето-преимущество с minimum-regret поведением (Claim 1) и routing strategy как per-family deployment decision (Claim 2); RQ4 опровергнут с интерпретацией universal negative finding (Claim 3). Главный structural finding работы — Claim 2: выбор стратегии маршрутизации в адаптивных мульти-агентных системах больших языковых моделей не является fixed-at-design-time гиперпараметром, а определяется характеристиками семейства весов рабочего агента. Это формулируется как методологическое требование — любые претензии на универсальность адаптивных политик маршрутизации в MAS LLM требуют валидации не менее чем на двух семействах весов работника. Universal negative по RQ4 дает дополнительный design-bound — достаточно одного уровня адаптивности (топологии); второй (роль) дает регрессию и должен быть исключен из базового дизайна.

Авторский вклад

Авторский вклад настоящей работы сводится к четырем результатам.

Первый — программная инфраструктура исследования. В рамках работы с нуля разработан и опубликован под открытой лицензией MIT мульти-агентный фреймворк адаптивных топологий для мульти-агентных систем (от англ. Adaptive Topologies for Multi-agent systems, ATM), объединяющий пять статических коммуникационных топологий, адаптивную мета-топологию второго уровня, конечный автомат фаз, маршрутизатор фаз и маршрутизатор топологий в двух режимах (правилковый и языковой), защитные правила переключения, шлюз взаимодействия с человеком с пятью ролями и маршрутизатор ролей, подсистему наблюдаемости с двухкомпонентным хранилищем (реляционная база PostgreSQL и колоночный формат Apache Parquet). Кодовая база содержит 21 327 строк кода на языке Python, распределенных по 112 программным модулям внутри тринадцати функциональных слоев.

Второй — формальная модель и таксономия метрик. Введена формализация мульти-агентной системы как кортежа из шести компонентов с зависящей от фазы коммуникационной структурой (формулы 2.2 и 2.3). Введена таксономия из пяти адаптивно-специфичных метрик — число переключений топологии, доля заблокированных решений маршрутизатора, стоимостный вклад маршрутизатора, доля регрессий качества и разрыв до оракула (формула 2.4). Введена прокси-метрика когнитивной нагрузки для запусков с симулятором (формула 2.5). Совокупность метрик позволяет количественно характеризовать поведение динамически перестраиваемых мульти-агентных систем и сопоставлять их по единой шкале.

Третий — эмпирические результаты. Проведена воронка из четырех основных экспериментов и двух кросс-семейных подтверждающих запусков общим объемом 4230 завершенных запусков (\$105,35); для каждого исследовательского вопроса получен явный количественный ответ (глава 5). RQ1 и RQ3 подтверждены; RQ2 раскладывается на task-difficulty-conditional Парето-преимущество (Claim 1) и routing strategy как per-family decision (Claim 2); RQ4 опровергнут с интерпретацией universal negative finding (Claim 3). Все головные эффекты сопровождаются непараметрическими бутстрап-доверительными интервалами; устойчиво исключают границу три из них.

Четвертый, ключевой — методологический вывод. Главный structural finding работы — выбор стратегии маршрутизации в адаптивных мульти-агентных системах больших языковых моделей является per-family deployment decision, а не fixed-at-design-time гиперпараметром. Адаптивные политики, выглядящие независимыми от базовой модели через таблицы правил и prompt-шаблоны, фактически зависят от нее через свои входы — фазовые классификаторы получают разные сигналы от работников разных семейств. На основании этого наблюдения сформулировано методологическое требование — любые претензии на универсальность адаптивных политик в мульти-агентных системах больших языковых моделей требуют валидации не менее чем на двух семействах весов рабочего агента. Это требование меняет статус кросс-семейной валидации с необязательного дополнения на обязательный шаг исследова-

ния адаптивных мульти-агентных систем. Дополнительно universal negative finding по RQ4 дает concrete design-bound — адаптивность на двух уровнях одновременно (топология и роль) избыточна, достаточно одного уровня (топологии); второй (роль) дает регрессию и должен быть исключен из базового дизайна.

Заключение

В работе проведена количественная оценка влияния выбора коммуникационной топологии, механизма ее адаптации и позиции человека на эффективность решения задач разных классов MAS LLM. Цель достигнута — получены количественные характеристики для пяти статических топологий, адаптивной мета-топологии в двух режимах маршрутизации, пяти ролей человека и их адаптивной комбинации; выполнено 4230 завершенных запусков на четырех корпусах задач с суммарной стоимостью \$105,35.

RQ1 (от англ. Research Question) подтвержден — универсальной статикидоминатора нет, победители на четырех корпусах различаются (Chain — HumanEval, Star — GSM8K, Debate — CommonGen и DABench). RQ3 подтвержден — HITL дает прирост +0,033 при неоднородности по ячейкам.

RQ2 раскладывается на task-difficulty-conditional Парето-преимущество на сложных корпусах (Claim 1) и инверсию Парето-победителя при переходе с gpt-oss-120b на Qwen-3-235B (Claim 2; см. §5.1). Выбор стратегии маршрутизации — per-family deployment decision. RQ4 опровергнут с интерпретацией universal negative finding (Claim 3) — адаптивный маршрутизатор роли universally не дает положительного лифта поверх адаптивной мета-топологии; эффект не реализационный, а в самой гипотезе двухуровневой адаптации.

Ключевой результат — routing strategy как per-family deployment decision в адаптивных мульти-агентных системах больших языковых моделей. Таблицы правил и архитектуры промптов, выглядящие независимыми от базовой модели, фактически зависят от нее через свои входы — фазовые классификаторы получают разные сигналы от работников разных семейств и относят те же ситуации к разным фазам. Это формирует методологическое требование к кросс-семейной валидации как обязательному шагу исследования. Дополнительно universal negative по RQ4 дает concrete design-bound — адаптивность на двух уровнях одновременно (топология и роль) избыточна, достаточно одного уровня (топологии).

Ограничения — $n = 45$ запусков на ячейку (корпус, режим) в E3 и E4 (на этом объеме устойчиво подтверждены три эффекта); кросс-семейная валидация на двух семействах рабочего агента; параметр глубины рассуждений смешан с эффектом семейства на Qwen; DABench дает model-sensitive ограничение на gpt-oss-120b (см. 4.6); человек моделируется языковой моделью; корпус не покрывает ряд классов (диалог, перевод, обобщение длинных текстов); судья — единственная модель из единственного семейства.

Направления дальнейших исследований

Прямое продолжение работы образует четыре взаимодополняющих направления. Первое — расширение кросс-семейной валидации на три и более семейства весов рабочего агента (Anthropic Claude, Google Gemini, Meta LLaMA-4) с приведением параметра глубины рассуждений к единой шкале, что устранил смещение эффектов семейства и режима работы агента и позволит проверить, является ли наблюдаемая кросс-семейная неустойчивость политик маршрутизации универсальным свойством адаптивных систем или специфичной для конкретной пары семейств. Второе — валидация симулятора человека на реальных участниках по схеме планируемого эксперимента E5 с использованием шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA), латинно-квадратного дизайна на восьми-двенадцати участниках и отобранного подмножества из двенадцати задач, что позволит численно оценить корреляцию между симулированной и фактической оценкой когнитивной нагрузки. Третье — разработка семейно-осознанного маршрутизатора, в котором политика выбора топологии и роли явно учитывает семейство весов рабочего агента, например, через предварительное дообучение маршрутизатора на трассах целевой модели или через переключение между семейно-специфичными политиками. Четвертое — расширение корпуса задач на непокрытые классы (диалог, перевод, обобщение длинных документов, мультимодальные задачи) и проверка обобщения результатов воронки на новые классы.

Библиографический список

- [1] A survey on large language model based autonomous agents / L. Wang, C. Ma, X. Feng [и др.] // *Frontiers of Computer Science*. – 2024. – Vol. 18, no. 6. – P. 186345.
- [2] AgentBench: Evaluating LLMs as agents [Электронный ресурс] / X. Liu, H. Yu, H. Zhang [и др.] // *arXiv preprint 2308.03688*. – 2023. – URL: <https://arxiv.org/abs/2308.03688> (дата обращения 17.05.2026).
- [3] AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors [Электронный ресурс] / W. Chen, Y. Su, J. Zuo [и др.] // *arXiv preprint 2308.10848*. – 2023. – URL: <https://arxiv.org/abs/2308.10848> (дата обращения: 17.05.2026).
- [4] AutoGen: Enabling next-gen LLM applications via multi-agent conversation [Электронный ресурс] / Q. Wu, G. Bansal, J. Zhang [и др.] // *arXiv preprint 2308.08155*. – 2023. – URL: <https://arxiv.org/abs/2308.08155> (дата обращения: 17.05.2026).
- [5] Benjamini Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing / Y. Benjamini, Y. Hochberg // *Journal of the Royal Statistical Society. Series B*. – 1995. – Vol. 57, no. 1. – P. 289–300.
- [6] CAMEL: Communicative agents for «mind» exploration of large language model society / G. Li, H. Hammoud, H. Itani, D. Khizbullin, B. Ghanem // *Advances in Neural Information Processing Systems*. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 51991–52008.
- [7] Chase H. LangChain: Building applications with LLMs through composability [Электронный ресурс] / H. Chase. – 2023. – URL: <https://github.com/langchain-ai/langchain> (дата обращения: 17.05.2026).
- [8] ChatDev: Communicative agents for software development / C. Qian, W. Liu, H. Liu [и др.] // *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. – Stroudsburg, PA: ACL, 2024. – P. 15174–15186.
- [9] Chen L. FrugalGPT: How to use large language models while reducing cost and improving performance [Электронный ресурс] / L. Chen, M. Zaharia, J. Zou // *Transactions on Machine Learning Research*. – 2024. – URL: <https://openreview.net/forum?id=cSimKw5p6R> (дата обращения: 17.05.2026).
- [10] Christiano P. F. Deep reinforcement learning from human preferences / P. F. Christiano, J. Leike, T. Brown [и др.] // *Advances in Neural Information Processing Systems*. – Red Hook, NY: Curran Associates, 2017. – Vol. 30. – P. 4299–4307.
- [11] CommonGen: A constrained text generation challenge for generative commonsense reasoning / B. Y. Lin, W. Zhou, M. Shen [и др.] // *Findings of the Association for Computational Linguistics (EMNLP 2020)*. – Stroudsburg, PA: ACL, 2020. – P. 1823–1840.
- [12] Cohen J. Statistical power analysis for the behavioral sciences / J. Cohen. – 2nd ed. – Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. – 567 p.
- [13] Colvin S. Pydantic: Data validation using Python type hints [Электронный ресурс] / S. Colvin [и др.]. – 2024. – URL: <https://docs.pydantic.dev/> (дата обращения: 17.05.2026).
- [14] Efron B. Bootstrap methods: another look at the jackknife / B. Efron // *The Annals of Statistics*. – 1979. – Vol. 7, no. 1. – P. 1–26.
- [15] Evaluating large language models trained on code [Электронный ресурс] / M. Chen, J. Tworek, H. Jun [и др.] // *arXiv preprint 2107.03374*. – 2021. – URL: <https://arxiv.org/abs/2107.03374> (дата обращения: 17.05.2026).
- [16] G-Designer: Architecting multi-agent communication topologies via graph neural networks [Электронный ресурс] / Y. Zhang, K. Xu, Y. Li, Z. Liu, D. Shen //

- arXiv preprint 2410.11782. – 2024. – URL: <https://arxiv.org/abs/2410.11782> (дата обращения 17.05.2026).
- [17] GPTSwarm: Language agents as optimizable graphs / M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, J. Schmidhuber // Proceedings of the International Conference on Machine Learning (ICML 2024). – Cambridge, MA: PMLR, 2024. – P. 62556–62583.
- [18] Hart S. G. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research / S. G. Hart, L. E. Staveland // Human Mental Workload / ed. by P. A. Hancock, N. Meshkati. – Amsterdam: North-Holland, 1988. – P. 139–183.
- [19] HellaSwag: Can a machine really finish your sentence? / R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, Y. Choi // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019). – Stroudsburg, PA: ACL, 2019. – P. 4791–4800.
- [20] Horvitz E. Principles of Mixed-Initiative User Interfaces / E. Horvitz // Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99). – New York: ACM, 1999. – P. 159–166.
- [21] Improving factuality and reasoning in language models through multiagent debate [Электронный ресурс] / Y. Du, S. Li, A. Torralba, J. Tenenbaum, I. Mordatch // arXiv preprint 2305.14325. – 2023. – URL: <https://arxiv.org/abs/2305.14325> (дата обращения: 17.05.2026).
- [22] InfiAgent-DABench: Evaluating agents on data analysis tasks [Электронный ресурс] / X. Hu, Z. Zhao, S. Wei [и др.] // arXiv preprint 2401.05507. – 2024. – URL: <https://arxiv.org/abs/2401.05507> (дата обращения: 17.05.2026).
- [23] Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation / J. Liu, C. S. Xia, Y. Wang, L. Zhang // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 21558–21572.
- [24] LangGraph: A framework for building stateful, multi-actor applications with LLMs [Электронный ресурс] / LangChain AI. – Электрон. дан. – 2024. – URL: <https://langchain-ai.github.io/langgraph/> (дата обращения 17.05.2026).
- [25] LangSmith: Observability and evaluation for LLM applications [Электронный ресурс] / LangChain AI. – Электрон. дан. – 2024. – URL: <https://docs.smith.langchain.com/> (дата обращения: 17.05.2026).
- [26] Lin C.-Y. ROUGE: A package for automatic evaluation of summaries / C.-Y. Lin // Text Summarization Branches Out: Proceedings of the ACL-04 Workshop. – Stroudsburg, PA: ACL, 2004. – P. 74–81.
- [27] LiveCodeBench: Holistic and contamination-free evaluation of large language models for code [Электронный ресурс] / N. Jain, K. Han, A. Gu [и др.] // arXiv preprint 2403.07974. – 2024. – URL: <https://arxiv.org/abs/2403.07974> (дата обращения: 17.05.2026).
- [28] Liu Z. Dynamic LLM-agent network: an LLM-agent collaboration framework with agent team optimization [Электронный ресурс] / Z. Liu, Y. Zhang, P. Li, Y. Liu, D. Yang // arXiv preprint 2310.02170. – 2023. – URL: <https://arxiv.org/abs/2310.02170> (дата обращения: 17.05.2026).
- [29] Magentic-One: A generalist multi-agent system for solving complex tasks [Электронный ресурс] / A. Fourney, G. Bansal, H. Mozannar [и др.] // arXiv preprint 2411.04468. – 2024. – URL: <https://arxiv.org/abs/2411.04468> (дата обращения: 17.05.2026).
- [30] Measuring mathematical problem solving with the MATH dataset / D. Hendrycks, C. Burns, S. Kadavath [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2021. – Vol. 34. – P. 2280–2292.

- [31] MetaGPT: Meta programming for a multi-agent collaborative framework / S. Hong, M. Zhuge, J. Chen [и др.] // Proceedings of the International Conference on Learning Representations (ICLR 2024). – 2024.
- [32] Mixture-of-Agents enhances large language model capabilities / J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, J. Zou // Proceedings of the Conference on Language Modeling (COLM 2024). – 2024.
- [33] MMLU-Pro: A more robust and challenging multi-task language understanding benchmark [Электронный ресурс] / Y. Wang, X. Ma, G. Zhang [и др.] // arXiv preprint 2406.01574. – 2024. – URL: <https://arxiv.org/abs/2406.01574> (дата обращения: 17.05.2026).
- [34] Mosqueira-Rey E. Human-in-the-loop machine learning: a state of the art / E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, Á. Fernández-Leal // Artificial Intelligence Review. – 2023. – Vol. 56, no. 4. – P. 3005–3054.
- [35] NetSafe: Exploring the topological safety of multi-agent networks against adversarial attacks [Электронный ресурс] / M. Yu, S. Wang, G. Zhang, H. Li // arXiv preprint 2410.15686. – 2024. – URL: <https://arxiv.org/abs/2410.15686> (дата обращения: 17.05.2026).
- [36] Ouyang L. Training language models to follow instructions with human feedback / L. Ouyang, J. Wu, X. Jiang [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2022. – Vol. 35. – P. 27730–27744.
- [37] ReAct: Synergizing reasoning and acting in language models / S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao // Proceedings of the International Conference on Learning Representations (ICLR 2023). – 2023.
- [38] Reflexion: Language agents with verbal reinforcement learning / N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 8634–8652.
- [39] RouteLLM: Learning to route LLMs with preference data [Электронный ресурс] / I. Ong, A. Almahairi, V. Wu [и др.] // arXiv preprint 2406.18665. – 2024. – URL: <https://arxiv.org/abs/2406.18665> (дата обращения: 17.05.2026).
- [40] Scaling large-language-model-based multi-agent collaboration [Электронный ресурс] / C. Qian, Z. Ye, W. Liu [и др.] // arXiv preprint 2406.07155. – 2024. – URL: <https://arxiv.org/abs/2406.07155> (дата обращения: 17.05.2026).
- [41] Self-Refine: Iterative refinement with self-feedback / A. Madaan, N. Tandon, P. Gupta [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 46534–46594.
- [42] Tipping the dominos: Topology-aware multi-hop attacks on LLM-based multi-agent systems [Электронный ресурс] / R. Liang, J. Ye, Z. Chen, Q. Zhu // arXiv preprint 2412.04129. – 2024. – URL: <https://arxiv.org/abs/2412.04129> (дата обращения: 17.05.2026).
- [43] Training verifiers to solve math word problems [Электронный ресурс] / K. Cobbe, V. Kosaraju, M. Bavarian [и др.] // arXiv preprint 2110.14168. – 2021. – URL: <https://arxiv.org/abs/2110.14168> (дата обращения: 17.05.2026).
- [44] Tree of Thoughts: Deliberate problem solving with large language models / S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, K. Narasimhan // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 11809–11822.

Приложение А Функциональная схема системы

В настоящем приложении приведена функциональная схема программной системы, описанной в главе 3. Схема изображает движение управления и данных от поступления входных данных эксперимента до получения результирующих метрик и журналов. На рисунке А.1 двойная красная рамка отмечает авторские компоненты — слой адаптации, оракул-маршрутизатор и шлюз взаимодействия с человеком (внутри блока «Внешние службы»). Штриховые красные стрелки — две обратные связи — оракульная таблица возвращается в маршрутизатор топологий, HITL-сигналы поступают от человека к слою адаптации.

Пояснение к схеме

Поток выполнения начинается с поступления конфигурации эксперимента в формате YAML и описания задачи $\mathcal{T}$ из выбранного корпуса. Оркестратор эксперимента (`atm.experiment`) инициализирует общее состояние графа и передает управление слою адаптации. Маршрутизатор фаз принимает решение о текущей фазе решения, передает результат маршрутизатору топологий, который выбирает активную коммуникационную топологию. В работе одновременно действует ровно одна из пяти базовых топологий, а адаптивный режим реализуется через последовательное переключение между ними по решению маршрутизатора.

Активная топология определяет порядок активации агентов и протокол обмена сообщениями между ними. Любая из шести ролей агентов (планировщик, исследователь, исполнитель, критик, дебатер, координатор) может быть подключена к любой топологии в соответствии с правилами раздела 2.2. Агенты обращаются за внешними службами — инструментами и Docker-песочницей для исполнения кода, оболочкой над языковыми моделями для генерации текста, шлюзом взаимодействия с человеком.

Все события системы (вызовы языковых моделей, выставление сигналов, переходы между фазами, переключения топологий, обращения к челове-

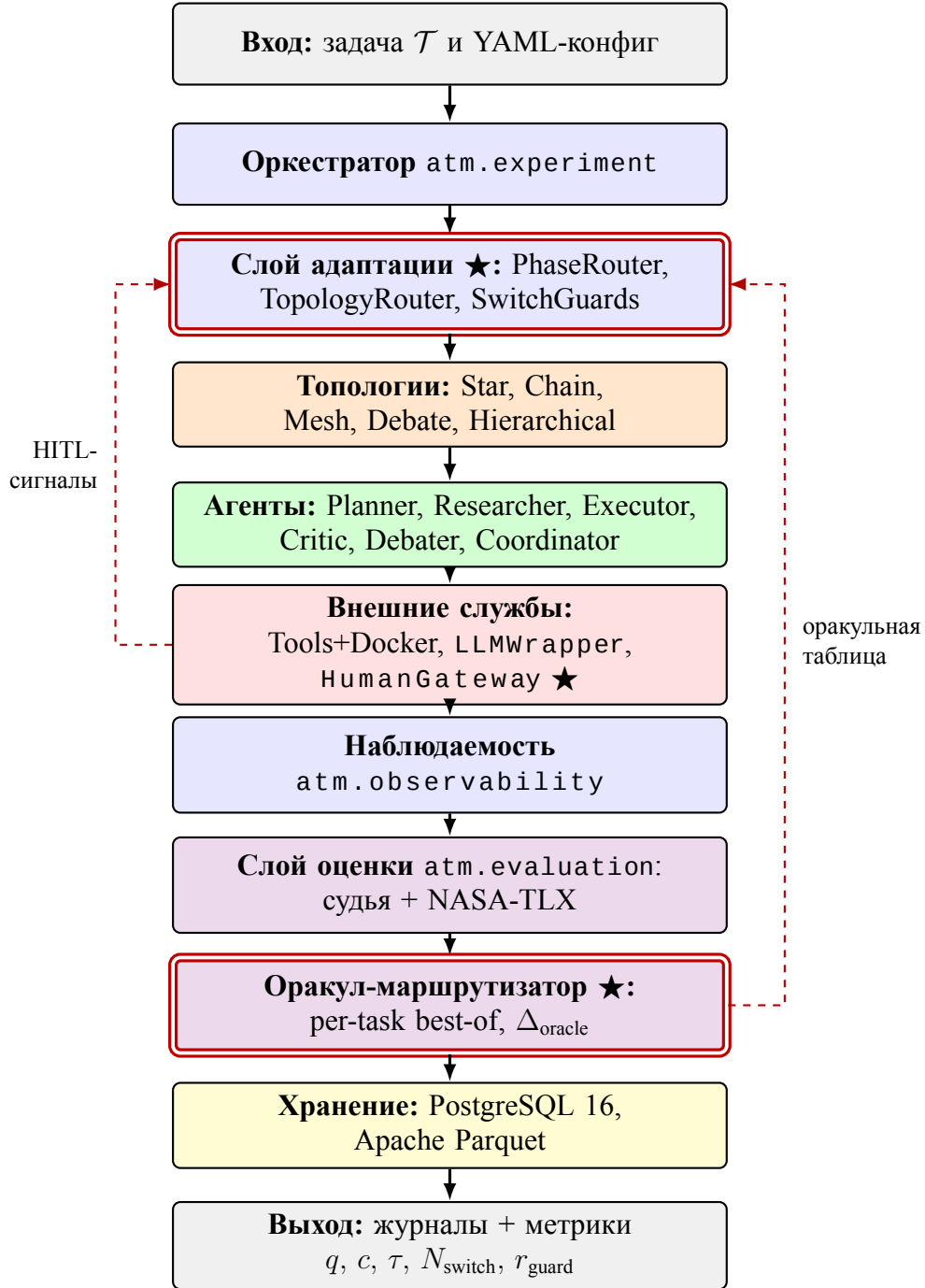


Рисунок А.1. Функциональная схема программной системы. Сплошные стрелки — прямой поток управления от входа к выходу. Штриховые красные стрелки — обратные связи — оракульная таблица возвращается в слой адаптации (маршрутизатор топологий), НITL-сигналы поступают от человека (внутри блока внешних служб) к слою адаптации. Двойная красная рамка обозначает авторские компоненты — слой адаптации, оракул-маршрутизатор и шлюз HumanGateway внутри блока внешних служб (отмечен ★). Полный перечень заимствованных и авторских компонентов с источниками приведен в таблице 3.2.

ку, срабатывания защитных правил маршрутизатора) перехватываются подсистемой наблюдаемости и направляются в два хранилища — реляционную базу для метаданных и колоночные файлы Apache Parquet для объемных журналов. На финальной стадии результаты запуска становятся доступными для постобработки слою анализа `atm.analysis`.

Обратный канал от шлюза человека к слою адаптации обозначает поступление управляющих сигналов от участника-человека. Через этот канал человек, действующий в роли координатора или наблюдателя, может форсировать переключение топологии или досрочно завершить фазу проверки. Штриховая связь от оракула к слою адаптации обеспечивает возврат оракульной таблицы (per-task best-of) для расчета Δ_{oracle} .

Приложение Б Глоссарий

В приложении приведены определения ключевых терминов работы, сгруппированные по тематическим блокам.

Мульти-агентные системы и языковые модели

LLM (Large Language Model) — глубокая нейронная сеть, обученная задаче языкового моделирования с дополнительной настройкой под инструкции. **MAS** (Multi-Agent System) — программная система из нескольких автономных агентов. **Агент** — программная сущность с фиксированной ролью, системной подсказкой, набором инструментов и собственным состоянием. **Скрэтчпад** — локальный контекст агента со скользящим окном истории и автоматическим сжатием при переполнении. **Судья** (LLM-as-judge) — внешняя языковая модель из другого семейства весов, оценивающая качество ответа с самосогласованностью. **Симулятор человека** — языковая модель в роли участника с подсказкой, зависящей от роли. **Бюджет вычислений** — трехуровневый ограничитель стоимости вызовов (на вызов / запуск / грид-эксперимент).

Топологии, фазы и маршрутизаторы

Коммуникационная топология — направленный граф допустимых маршрутов обмена сообщениями между агентами. **Star / Chain / Mesh / Debate / Hierarchical** — пять базовых топологий (центр-периферия, конвейер, полносвязная, два оппонента и судья, двухуровневая иерархия). **Адаптивная мета-топология** — конфигурация с выбором активной базовой топологии на каждой итерации в зависимости от фазы и сигналов; авторская разработка. **FSM** (Finite-State Machine) — конечный автомат фаз решения (планирование, исполнение, проверка, завершение). **PhaseRouter, TopologyRouter, RoleRouter** — маршрутизаторы фаз, топологий и ролей человека в трех режимах — правилый, на основе языковой модели и оракульный. **SwitchGuards** — защитные правила переключения против частого осцилляционного режима (min_dwell, cooldown, лимиты за фазу и запуск).

Человек в контуре управления

HITL (Human-in-the-Loop) — парадигма обращения автоматизированной системы к человеку за решением, оценкой или подтверждением. **HumanGateway** — протокол шлюза с идемпотентностью по паре идентификаторов запуска и запроса. **Пять ролей человека** — координатор (управляет планом), рецензент (проверяет результаты), судья (выносит вердикт), равноправный участник (вносит сообщения наравне с агентами), наблюдатель (вмешивается только в критических случаях). **NASA-TLX** — стандартная шкала субъективной оценки

когнитивной нагрузки по шести подшкалам.

Метрики

q — агрегированное качество в $[0, 1]$, формула зависит от класса задач. c — суммарная стоимость вызовов в долларах за запуск. τ — полное время запуска по настенным часам. N_{switch} — число фактических переключений активной топологии за запуск. r_{guard} — доля решений маршрутизатора, заблокированных защитными правилами. γ — доля бюджета на собственные вызовы маршрутизатора. r_{hurt} — доля задач с качеством хуже лучшей статической конфигурации. Δ_{oracle} — разрыв между оракул-выбором лучшей статической топологии для каждого корпуса (per-task best-of) и фактическим маршрутизатором. L_{cog} — прокси-метрика когнитивной нагрузки из числа обращений, объема контекста и задержки ответа симулятора.

Инфраструктура и эксперимент

LangGraph — библиотека Python для построения направленных графов состояния с чекпойнтером. **Apache Parquet** — колоночный формат хранения структурированных данных, используется для журналов взаимодействия. **Запуск (run)** — один цикл выполнения мульти-агентной системы; элементарная единица измерения. **Грид-эксперимент** — серия запусков по декартову произведению параметров. **Воронка экспериментов** — последовательность экспериментов с сужением пространства конфигураций по результатам предыдущих. **Подтверждающий запуск** — эксперимент на отличной языковой модели для проверки независимости выводов от семейства весов. **Бутстрап** — непараметрический метод оценивания распределения статистики через ресэмплирование; в работе применяется метод процентилей с десятью тысячами ресэмплов на уровне 95 %. **Поправка Бенджамини–Хохберга** — процедура контроля доли ложных открытий при множественном сравнении. **Коэффициент Коэна (d)** — стандартизированная мера величины различия двух средних.

Приложение В Состав программного репозитория

Состав репозитория фиксирован по идентификатору фиксации приложения Г.

Корневая структура

src/atm/ — основной пакет (13 слоев). **conf/** — YAML-конфигурации экспериментов и компонентов. **scripts/** — скрипты обслуживания. **notebooks/** — шаблон Jupyter-ноутбука для постобработки. **pyproject.toml** — декларация пакета и точки входа **atm**. **README.md** — инструкция по сборке.

Слой пакета **src/atm/**

atm.core — состояние и базовые типы. **atm.llm** — оболочка провайдеров, ценообразование, бюджеты, FakeLLM. **atm.tools** — реестр инструментов и Docker-песочница. **atm.agents** — класс Agent, пять ролей и координатор. **atm.topology** — шесть топологий поверх протокола Topology. **atm.phases** — FSM фаз, маршрутизатор топологий, защитные правила, ворота передачи состояния. **atm.human** — HITL: HumanGateway, шлюзы, маршрутизатор ролей, политика тайм-аутов. **atm.storage** — ORM, асинхронные сессии PostgreSQL, писатель Parquet, чекпойнтер, миграции alembic/. **atm.observability** — перехватчик событий LangGraph. **atm.tasks** — загрузчики и оценщики четырех корпусов. **atm.evaluation** — судья качества, оракулы, агрегатор NASA-TLX. **atm.experiment** — схемы Pydantic, OmegaConf, CLI на Typer. **atm.analysis** — агрегаторы метрик, графики, построитель оракула.

Конфигурации **conf/**

experiments/ — конфиги E1–E4 и подтверждающих запусков. **agents/**, **topology/**, **human/**, **model/**, **tasks/**, **oracle/**, **evaluation/**, **sandbox/** — параметры по соответствующим компонентам.

Приложение Г Открытый исходный код

В соответствии с пунктом 5.5 Правил подготовки и защиты ВКР полная программная реализация настоящего исследования размещена в открытом доступе на платформе GitHub.

Адрес репозитория

<https://github.com/cactus-tim/adaptive-topologies-mas>

Идентификатор фиксации

ветка `main`, короткий идентификатор коммита `058271a39a0e`, полный хеш:

`058271a39a0e4ecc0e3b47bd1d826c03fc33872b`.

Лицензия

MIT (файл `LICENSE`).

Сборка и воспроизведение

инструкция в `README.md`; конфигурации экспериментов главы 4 — в директории `conf/experiments/`; запуск одиночного запуска — `atm run --config <file>`, грид-эксперимента — `atm grid --config <file>`, где `<file>` — путь к YAML-конфигу.

Согласие автора на публикацию ВКР на портале НИУ ВШЭ (п. 5.6.3 Правил) оформлено QR-кодом системы «LMS-Антиплагиат» в составе пакета документов в ГЭК.